

Differentially Private Compression and the Sensitivity of LZ77

Jeremiah Blocki¹[0000–0002–5542–4674], Seunghoon Lee^{*2}[0000–0003–4475–5686],
and Brayan Sebastián Yepes-García³[0009–0002–9162–390X]

¹ Purdue University, West Lafayette, USA
jblocki@purdue.edu

² University of Waterloo, Waterloo, Canada
seunghoon.lee@uwaterloo.ca

³ Universidad Nacional de Colombia, Bogotá, Colombia
byepesg@unal.edu.co

Abstract. We initiate the study of differentially private data-compression schemes motivated by the insecurity of the popular “Compress-Then-Encrypt” framework. Data compression is a useful tool which exploits redundancy in data to reduce storage/bandwidth when files are stored or transmitted. However, if the contents of a file are confidential then the *length* of a compressed file might leak confidential information about the content of the file itself. Encrypting a compressed file does not eliminate this leakage as data encryption schemes are only designed to hide the *content* of confidential message instead of the *length* of the message. In our proposed *Differentially Private Compress-Then-Encrypt* framework, we add a random positive amount of padding to the compressed file to ensure that any leakage satisfies the rigorous privacy guarantee of (ϵ, δ) -differential privacy. The amount of padding that needs to be added depends on the sensitivity of the compression scheme to small changes in the input, i.e., to what degree can changing a single character of the input message impact the length of the compressed file. While some popular compression schemes are highly sensitive to small changes in the input, we argue that effective data compression schemes do not necessarily have high sensitivity. Our primary technical contribution is analyzing the fine-grained sensitivity of the LZ77 compression scheme (IEEE Trans. Inf. Theory 1977) which is one of the most common compression schemes used in practice. We show that the global sensitivity of the LZ77 compression scheme has the upper bound $\mathcal{O}(W^{2/3} \log n)$ where $W \leq n$ denotes the size of the sliding window. When $W = n$, we show the lower bound $\Omega(n^{2/3} \log^{1/3} n)$ for the global sensitivity of the LZ77 compression scheme which is tight up to a sublogarithmic factor.

1 Introduction

Data compression algorithms exploit redundancy in data to compress files before storage or transmission. Lossless compression schemes are widely used in

^{*} Work done primarily while Seunghoon was at Purdue University.

contexts such as ZIP archives (Deflate [8]) and web content compression by Google (Brotli [2]). In the Compress-Then-Encrypt framework, a message w is first compressed to obtain a (typically shorter) file $y = \text{Compress}(w)$ and then encrypted to obtain a ciphertext $c = \text{E}_K(y)$. Intuitively, compression is used to reduce bandwidth, and encryption is used to protect the file contents from eavesdropping attacks before it is transmitted over the internet.

While encryption protects the *content* of the underlying plaintext message y , there is no guarantee that the ciphertext c will hide the *length* of the underlying plaintext y . For example, AES-GCM [4] is the most widely used symmetric key encryption scheme, but the ciphertext leaks the *exact* length of the underlying plaintext⁴. Unfortunately, the length of the compressed message y can leak information about the *content* of the original uncompressed file. For example, suppose that the original message was $w = \text{"The top secret password is: password."}$. An eavesdropping attacker who intercepts the AES-GCM ciphertext c would be able to infer the length $|\text{Compress}(w)|$ and could use this information to eliminate many incorrect password guesses, i.e., if $|\text{Compress}(w')| \neq |\text{Compress}(w)|$ where $w' = \text{"The top secret password is: [guess]}"$ then the attacker could immediately infer that this particular password guess is incorrect.

The observation that the length of a compressed file can leak sensitive information about the *content* has led to several real-world exploits. For example, the CRIME (Compression Ratio Info-leak Made Easy) attack [23] exploits compression length leakage to hijack TLS sessions [11]. Similarly, the BREACH (Browser Reconnaissance and Exfiltration via Adaptive Compression of Hypertext) attack [13] against the HTTPS protocol exploits compression in the underlying HTTP protocol. Despite its clear security issues, the “Compress-then-Encrypt” framework continues to be used as a tool to reduce bandwidth overhead. There have been several heuristic proposals to address this information leakage. One proposal [21] is static analysis to identify/exclude sensitive information before data compression. However, it is not always clear *a priori* what information should be considered sensitive. Heal the Breach [20] pads compressed files by a random amount to mitigate leakage and protect against the CRIME/BREACH attacks. While this defense is intuitive, there will still be some information leakage and there are no rigorous privacy guarantees. This leads to the following question:

Can one design a compression scheme which provides rigorous privacy guarantees against an attacker who learns the compressed file’s length?

Differential Privacy (DP) [9] has emerged as a gold standard in privacy-preserving data analysis due to its rigorous mathematical guarantees and strong composition results. Given $\epsilon > 0$ and $\delta \in (0, 1)$, a randomized algorithm $\mathcal{A}$ is called (ϵ, δ) -*differentially private* if for every pair of neighboring datasets $\mathcal{D}$ and $\mathcal{D}'$ and for all sets S of possible outputs, we have $\Pr[\mathcal{A}(\mathcal{D}) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{A}(\mathcal{D}') \in S]$.

⁴ Any efficient encryption scheme must at least leak some information about the length of the underlying plaintext. Otherwise, the ciphertext of a short message (e.g., “hi”) would need to be longer than the longest possible message supported by the protocol (e.g., a 4GB movie).

$S] + \delta$. Typically, ϵ is a small constant and the additive loss δ should be negligible (if $\delta = 0$ then we can simply say that $\mathcal{A}$ satisfies ϵ -DP). In our context, we are focused on leakage from the length of a compressed file. We say that a compression algorithm **Compress** is (ϵ, δ) -*differentially private* if for every pair of neighboring strings w and w' (e.g., differing in one symbol) and all $S \subseteq \mathbb{N}$ we have $\Pr[|\text{Compress}(w)| \in S] \leq e^\epsilon \cdot \Pr[|\text{Compress}(w')| \in S] + \delta$. Intuitively, suppose that w denotes the original message and w' denotes the message after replacing a sensitive character by a special blank symbol. Differential privacy ensures that an attacker who observes the length of the compressed file will still be unlikely to distinguish between w and w' . DP composition ensures that the attacker will also be unlikely to distinguish between the original string w and a string w'' where *every* symbol of a secret password has been replaced — provided that the secret password is not too long.

We first note that the Heal the Breach proposal [20] does not necessarily satisfy differential privacy. For example, Lagarde and Perifel [15] proved that the LZ78 compression algorithm [17] is highly sensitive to small bit changes. In particular, there exists nearby strings $w \sim w'$ (i.e., $|w| = |w'|$ and $|\{i : w[i] \neq w'[i]\}| = 1$) of length n such that $|\text{Compress}_{\text{LZ78}}(w)| = o(n)$ and $|\text{Compress}_{\text{LZ78}}(w')| = \Omega(n)$. In particular, to prevent an eavesdropping attacker from distinguishing between w and w' , the length of the padding would need to be at least $\Omega(n)$. Heal the Breach [20] adds a much smaller amount of padding $o(n)$.

Notations. For a positive integer n , we define $[n] := \{1, \dots, n\}$. Unless otherwise noted, we will use n as the length of a string throughout the paper. We use Σ to denote a set of alphabets and $w \in \Sigma^n$ to denote a string of length n and we let $w[i] \in \Sigma$ denote the i th character of the string. Given two strings $w, w' \in \Sigma^n$, we use $\text{Ham}(w, w') = |\{i : w[i] \neq w'[i]\}|$ to denote the *hamming distance* between w and w' , i.e., the number of indices where the strings do not match. We say that two strings w and w' are *adjacent* (or *neighbors*) if $\text{Ham}(w, w') = 1$ and we use $w \sim w'$ to denote this. We use $|w|$ to denote the length of a string, i.e., for any string $w \in \Sigma^n$ we have $|w| = n$. We use $w \circ w'$ to denote the concatenation of two strings w and w' . Given a character $c \in \Sigma$ and an integer $k \geq 1$, we use c^k to denote the character c repeated k times. Formally, we define $c^1 := c$ and $c^{i+1} := c \circ c^i$. Unless otherwise noted, we assume that all log's have base 2, i.e., $\log x := \log_2 x$.

1.1 Our Contributions

In this paper, we initiate the study of differentially private compression schemes focusing especially on the LZ77 compression algorithm [16]. We first provide a general transformation showing how to make any compression scheme differentially private by adding a random amount of padding which depends on the global sensitivity (the worst-case difference in the compressed length when one input symbol is modified; see [Definition 2](#) for the formal definition) of the compression scheme. The transformation will yield an efficient compression scheme as long as the underlying compression scheme has low global sensitivity. Second, we

demonstrate that good compression schemes do not inherently have high global sensitivity by demonstrating that Kolmogorov compression has low global sensitivity. Third, we analyze the global sensitivity of the LZ77 compression scheme providing an upper/lower bound that is tight up to the sublogarithmic factor of $\log^{2/3} n$.

1.1.1 Differentially Private Transform for Compression Algorithms.

We provide a general framework to transform any compression scheme **Compress** into a differentially private compression algorithm $\text{DPCompress}(w, \epsilon, \delta)$ by adding a random positive amount of padding to $\text{Compress}(w)$, where the amount of padding p depends on the privacy parameters ϵ and δ as well as the global sensitivity of the underlying compression algorithm **Compress**. More concretely, we show that $A_{\epsilon, \delta}(w) := \text{DPCompress}(w, \epsilon, \delta)$ is (ϵ, δ) -differentially private, i.e., the length leakage is (ϵ, δ) -differentially private. The expected amount of padding added is $\mathcal{O}\left(\frac{\text{GS}_{\text{Compress}}}{\epsilon} \ln\left(\frac{1}{2\delta}\right)\right)$ (see [Definition 2](#) for the definition of the global sensitivity $\text{GS}_{\text{Compress}}$). Thus, as long as $\text{GS}_{\text{Compress}} \ll n$, it is possible to achieve a compression ratio $|\text{DPCompress}(w, \epsilon, \delta)|/n = |\text{Compress}(w)|/n + o(1)$ that nearly matches **Compress**. See [Section 3](#) for details.

1.1.2 Idealized Compression Schemes have Low Sensitivity.

While the LZ78 [\[17\]](#) compression algorithm has high global sensitivity, we observe that compression schemes achieving optimal compression ratios do not inherently have high global sensitivity. In particular, we argue that Kolmogorov compression has low global sensitivity, i.e., at most $\mathcal{O}(\log n)$ where n denotes the string length parameter. While Kolmogorov compression is uncomputable, it is known to achieve a compression rate that is *at least as good* as any compression algorithm. We also construct a *computable* variant of Kolmogorov compression that preserves the global sensitivity, i.e., the global sensitivity of the computable variant of Kolmogorov compression is also $\mathcal{O}(\log n)$. See [Section 6](#) for details.

1.1.3 Global Sensitivity of the LZ77 Compression Scheme.

Our primary technical contribution is to analyze the global sensitivity of the LZ77 compression algorithm [\[16\]](#). The LZ77 algorithm includes a tunable parameter $W \leq n$ for the size of the sliding window. Selecting smaller W reduces the algorithm's space footprint, but can result in worse compression rates because the algorithm can only exploit redundancy within this window. We provide an almost tight upper and lower bound of the global sensitivity of the LZ77 compression scheme. In particular, we prove that the global sensitivity of the LZ77 compression scheme for strings of length n is upper bounded by $\mathcal{O}(W^{2/3} \log n)$ where $W \leq n$ is the length of the sliding window. When $W = n$ the global sensitivity is lower bounded by $\Omega(n^{2/3} \log^{1/3} n)$ matching our upper bound up to a sublogarithmic factor of $\log^{2/3} n$. We hope that our initial paper inspires follow-up research analyzing the global sensitivity of other compression schemes.

Theorem 1 (informal). *Let LZ77 be the LZ77 compression scheme for strings of length n with a sliding window size $W \leq n$. Then $\text{GS}_{\text{LZ77}} = \mathcal{O}(W^{2/3} \log n)$ and, when $W = n$, $\text{GS}_{\text{LZ77}} = \Omega(n^{2/3} \log^{1/3} n)$. (See [Theorem 3](#) and [Theorem 5](#).)*

At a high level, the upper bound analysis considers the relationship between the blocks generated by running the LZ77 compression for two neighboring strings $w \sim w'$. Let $B_1, \dots, B_t$ (resp. $B'_1, \dots, B'_{t'}$) denote the blocks generated when we run LZ77 with input w (resp. w') where block B_i (resp. B'_k) encodes the substring $w[s_i, f_i]$ (resp. $w'[s'_k, f'_k]$). We prove that if $s_i \leq s'_k \leq f_i$ (block B'_k starts inside block B_i) then $f'_{k+1} \geq f_i$. In particular, this means that for every block B_i there are *at most two* blocks from $B'_1, \dots, B'_{t'}$ that “start inside” B_i . We can then argue that the difference in compression lengths is proportional to t_2 where $t_2 = |\{i \leq t : |\{k \leq t' : s_i \leq s'_k \leq f_i\}| = 2\}|$ denotes the number of “type-2” blocks, i.e., B_i which have two blocks that start inside it. Finally, we can upper bound $t_2 = \mathcal{O}(n^{2/3})$ when $W = n$ or $t_2 = \mathcal{O}(W^{2/3})$ when $W < n$.

The lower bound works by constructing a string which (nearly) maximizes t_2 . See [Section 4](#) and [Section 5](#) for details.

1.2 Related Work

There has been extensive work on developing efficient compression algorithms. Huffman coding [\[14\]](#) encodes messages symbol by symbol — symbols that are used most frequently are encoded by shorter binary strings in the prefix-free code. Lempel and Ziv [\[16, 17\]](#) developed multiple algorithms, and Welch [\[29\]](#) later proposed LZW as an improved implementation of [\[17\]](#). Our work primarily focuses on analyzing the LZ77 compression algorithm [\[16\]](#) since it is one of the most common lossless compression algorithms used in practice. Deflate [\[8\]](#) is a lossless compression scheme that combines LZ77 compression [\[16\]](#) and Huffman coding [\[14\]](#) and is a key algorithm used in ZIP archives. The Lempel–Ziv–Markov chain algorithm (LZMA) is used in the 7z format of the 7-Zip archiver and it uses a modification of the LZ77 compression. Brotli [\[2\]](#) also uses the LZ77 compression with Huffman coding and the 2nd-order context modeling.

Several prior papers have studied the sensitivity of compression schemes [\[15, 1, 12\]](#) to small input changes. Lagarde and Perifel [\[15\]](#) studied the multiplicative sensitivity of the LZ78 compression algorithm [\[17\]](#), and their result implies that the additive (i.e., global) sensitivity can be as large as $\Omega(n)$. Giuliani et al. [\[12\]](#) showed that the additive sensitivity of the Burrows–Wheeler Transform with Run-Length Encoding can be as large as $\Omega(\sqrt{n})$ — upper bounding the additive sensitivity remains an open question.

Most closely related to ours is the work of Akagi et al. [\[1\]](#) who studied the additive and multiplicative sensitivity of several compression schemes including Kolmogorov and LZ77. Akagi et al. [\[1\]](#) proved that Kolmogorov compression has additive sensitivity $\mathcal{O}(\log n)$. We extend this result to a computable variation of Kolmogorov compression in [Section 6](#). For LZ77, they proved that the additive sensitivity is lower bounded by $\Omega(\sqrt{n})$ when the window size is $W = \Omega(n)$ ⁵. We prove that the additive sensitivity is lower bounded by $\tilde{\Omega}(n^{2/3})$. Finally, they

⁵ Bannai et al. [\[3\]](#) also lower bounds the additive sensitivity of LZ77 by $\Omega(\sqrt{n})$ as well though the primary focus in [\[3\]](#) is on efficiently updating the compressed file after rotations instead of analyzing additive sensitivity.

prove that the (local) additive sensitivity of LZ77 is *at most* $\mathcal{O}(z)$ where z is the length of the compressed file. Unfortunately, this result does not even imply that the global sensitivity is $o(n)$ because z can be as large as $z = \Omega(n)$ when the file is incompressible. By contrast, we prove that the global sensitivity is upper bounded by $\mathcal{O}(W^{2/3} \log n)$ which is tight up to logarithmic factors and is at most $\mathcal{O}(n^{2/3} \log n)$ even when $W = \Omega(n)$.

Degabriele [7] introduced a formal model for length-leakage security of compressed messages with random padding. While Degabriele [7] did not use differential privacy as a security notion, his analysis suggests that DP friendly distributions such as the Laplace distribution and Gaussian distribution minimized the leakage. Song [25] analyzed the (in)security of compression schemes against attacks such as cookie-recovery attacks and used the additive sensitivity of a compression scheme to upper bound the probability of successful attacks. Neither work [7, 25] formalized the notion of a differentially private compression scheme as we do in [Section 3](#).

Ratliff and Vadhan introduced a framework for differential privacy against timing attacks [22] to deal with information leakage that could occur when the running time of a (randomized) algorithm might depend on the sensitive input. They propose to introduce random positive delays after an algorithm is finished. The length of this delay will depend on the sensitivity of the algorithm's running time to small changes in the input. While our motivation is different, there are similarities: they analyze the sensitivity of an algorithm's running time while we analyze the sensitivity of a compression algorithm with respect to the length of the file. They introduce a random positive delay while proposing to add a random positive amount of padding to the compressed file.

2 Preliminaries

Definition 1 (Lossless Compression). *Let Σ, Σ' be sets of alphabets. A lossless compression scheme consists of two functions $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ and $\text{Decompress} : (\Sigma')^* \rightarrow \Sigma^*$. We require that for all string $w \in \Sigma^*$ we have $\text{Decompress}(\text{Compress}(w)) = w$.*

Global Sensitivity of Compression Scheme. Global sensitivity helps us understand how much the output of a function can change when its input is slightly modified. Recall that two strings $w, w' \in \Sigma^n$ are neighbors (denoted $w \sim w'$) if the Hamming Distance between these two strings is at most one, i.e., $|\{i : w[i] \neq w'[i]\}| \leq 1$. In the context of compression schemes, the global sensitivity of a compression scheme is defined as the largest difference in the compression length when we compress two neighboring strings $w \sim w'$.

Definition 2 (Local/Global Sensitivity). *Let Σ, Σ' be sets of alphabets. The local sensitivity of the compression algorithm $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ at $w \in \Sigma^n$ is defined as $\text{LS}_{\text{Compress}}(w) := \max_{w' \in \Sigma^n : w \sim w'} ||\text{Compress}(w)| - |\text{Compress}(w')||$, and the global sensitivity of the compression algorithm for strings of length n*

is defined as $\text{GS}_{\text{Compress}}(n) := \max_{w \in \Sigma^n} \text{LS}_{\text{Compress}}(w)$. If it is clear from context, then one can omit the parameter n (length of a string) and simply write $\text{GS}_{\text{Compress}}$ to denote the global sensitivity.

Understanding the global sensitivity of a compression algorithm is a crucial element of designing a differentially private compression scheme. While there is a large line of work using global sensitivity [24, 10, 30, 27, 18, 31, 26, 28, 19, 6, 5] to design differentially private mechanisms, to the best of our knowledge, no prior work has studied the construction of differentially private compression schemes.

In our context, the amount of random padding added to a compressed file will scale with the global sensitivity of the compression algorithm. While Lagarde and Perifel [15] were not motivated by privacy or security, their analysis of LZ78 implies that the global sensitivity of this compression algorithm is $\Omega(n)$. Thus, to achieve differential privacy, the random padding would need to be very long, i.e., at least $\Omega(n)$. This would immediately negate any efficiency gains since, after padding, the compressed file would not be shorter than the original file!

LZ77 Compression Scheme. The LZ77 compression algorithm [16] takes as input a string $w \in \Sigma^n$ and outputs a sequence of blocks $B_1, \dots, B_k$ where each block $B_i = [q_i, \ell_i, c_i]$ is a tuple consisting of two non-negative integers q_i and ℓ_i and a character $c_i \in \Sigma$. Intuitively, if blocks $B_1, \dots, B_{i-1}$ encode the first ctc characters of w then the block $B_i = [q_i, \ell_i, c_i]$ encodes the next $\ell_i + 1$ characters of w . In particular, if we have already recovered $w[1, \text{ctc}]$ then the decoding algorithm can recover the substring $w[\text{ctc} + 1, \text{ctc} + \ell_i + 1] = w[q_i, q_i + \ell_i - 1] \circ c_i$ can be recovered by copying ℓ_i characters from $w[1, \text{ctc}]$ beginning at index q_i and then appending the character c_i . The compression algorithm defines the first block as $B_1 = [0, 0, w[1]]$ (where $w[i]$ denotes the i^{th} character of w) and then initializes counters $\text{ctb} = 2$ and $\text{ctc} = 1$. Intuitively, the counter ctb indicates that we will output block B_{ctb} next and the parameter ctc counts the numbers of characters of w that have already been encoded by blocks $B_1, \dots, B_{\text{ctb}-1}$. In general, to produce the ctb^{th} block we will find the longest prefix of $w[\text{ctc} + 1, n]$ that is a substring of $w[\max\{1, \text{ctc} - W + 1\}, \text{ctc}]$, where $W \leq n$ denotes the size of the sliding window — the algorithm only stores W most recent characters to save space. If the character $w[\text{ctc} + 1]$ is not contained in $w[\max\{1, \text{ctc} - W + 1\}, \text{ctc}]$ then we will simply set $B_{\text{ctb}} = [0, 0, w[\text{ctc} + 1]]$ and increment the counters ctb and ctc . Otherwise, suppose that the longest such prefix has length $\ell_{\text{ctb}} > 0$ then for some z such that $\max\{0, \text{ctc} - W\} + \ell_{\text{ctb}} \leq z \leq \text{ctc}$ we have $w[\text{ctc} + 1, \text{ctc} + \ell_{\text{ctb}}] = w[z - \ell_{\text{ctb}} + 1, z]$. Then we set $B_{\text{ctb}} = [z - \ell_{\text{ctb}} + 1, \ell_{\text{ctb}}, w[\text{ctc} + \ell_{\text{ctb}} + 1]]$, increment ctb , and update $\text{ctc} = \text{ctc} + \ell_{\text{ctb}} + 1$. We terminate the algorithm if $\text{ctc} = n$ (i.e., the entire string has been encoded) and then output the blocks $(B_1, \dots, B_{\text{ctb}-1})$. See Figure 1 for a toy example of running the LZ77 compression for a string $w = \text{"aababcbabca"} \in \Sigma^{12}$ with $\Sigma = \{a, b, c, d\}$ and $W = n$.

The decompression works straightforwardly as follows: (1) Given the compression $(B_1, \dots, B_t)$, parse $B_1 = [0, 0, c_1]$ and initialize the string $w = \text{"}c_1\text{"}$. (2) For $i = 2$ to t , parse $B_i = [q_i, \ell_i, c_i]$ and convert it into a string v : if $q_i = \ell_i = 0$ then $v := c_i$; otherwise, $v := w[q_i, q_i + \ell_i - 1] \circ c_i$. Then update $w \leftarrow w \circ v$.

$$w \quad \begin{array}{c} B_1 \quad B_2 \quad B_3 \quad B_4 \quad B_5 \\ \boxed{a} \quad \boxed{ab} \quad \boxed{abc} \quad \boxed{d} \quad \boxed{babca} \end{array}$$

Initialize: $B_1 = [0, 0, a]$, $\text{ctb} = 2$, and $\text{ctc} = 1$

Intermediate Steps:

ctb	ctc	ℓ_{ctb}	z	B_{ctb}	updated ctb	updated ctc
2	1	1	1	$[1, 1, b]$	3	3
3	3	2	3	$[2, 2, c]$	4	6
4	6	0	N/A	$[0, 0, d]$	5	7
5	7	4	6	$[3, 4, a]$	6	12

Output: $(B_1 = [0, 0, a], B_2 = [1, 1, b], B_3 = [2, 2, c], B_4 = [0, 0, d], B_5 = [3, 4, a])$

Figure 1. Example of Running the LZ77 Compression for a string $w = \text{"aababcbabca"}$.

The LZ77 compression scheme described above is called the *non-overlapping* (i.e., *without self-referencing*) LZ77. In the non-overlapping LZ77, it is always the case that $q_i + \ell_i - 1 \leq \text{ctc}$ (so that the substring that we copy from the previous encoding does not overlap with the current encoding). We still obtain a valid compression scheme if we allow for the case $q_i + \ell_i - 1 > \text{ctc}$, and we refer to this scheme as LZ77 compression with self-referencing. We focus on the non-overlapping LZ77 in the main body of this paper. However, as we demonstrate in [Appendix B.2](#), our techniques extend to LZ77 with self-referencing.

3 Differentially Private Compression

This section presents a general framework that transforms any compression scheme $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ to another compression scheme called $\text{DPCompress} : \Sigma^* \times \mathbb{R} \times \mathbb{R} \rightarrow (\Sigma')^*$ such that for any $\epsilon > 0$ and $\delta \in (0, 1)$, the algorithm $A_{\epsilon, \delta}(w) := \text{DPCompress}(w, \epsilon, \delta)$ is a (ϵ, δ) -differentially private compression algorithm as defined formally in [Definition 3](#).

Definition 3. We say that a randomized compression algorithm $A : \Sigma^* \rightarrow (\Sigma')^*$ is (ϵ, δ) -differentially private if for every $n \geq 1$, every pair of neighboring strings $w \sim w' \in \Sigma^n$ and all $S \subseteq \mathbb{N}$ we have

$$\Pr[|A(w)| \in S] \leq e^\epsilon \cdot \Pr[|A(w')| \in S] + \delta.$$

Intuitively, $\text{DPCompress}(w, \epsilon, \delta)$ works by running $\text{Compress}(w)$ and adding a random amount of padding. In particular, $\text{DPCompress}(w, \epsilon, \delta)$ outputs $\text{Compress}(w) \circ 0 \circ 1^{p-1}$ where p is a random variable defined as follows: $p = \max\{1, \lceil Z + k \rceil\}$: where $Z \sim \text{Lap}(\text{GS}_{\text{Compress}}/\epsilon)$ is a random variable following Laplace distribution with mean 0 and scale parameter $\text{GS}_{\text{Compress}}/\epsilon$ and $k = \frac{\text{GS}_{\text{Compress}}}{\epsilon} \ln(\frac{1}{2\delta}) + \text{GS}_{\text{Compress}} + 1$ is a constant (see [Algorithm 1](#)). The decompression algorithm works straightforwardly as it is easy to remove padding $0 \circ 1^{p-1}$ even though

one does not know p , i.e., given the compressed string $\text{DPCompress}(w, \epsilon, \delta) = \text{Compress}(w) \circ 0 \circ 1^{p-1}$, we could start removing 1's from the right until we see 0. After removing the 0 as well, we get $\text{Compress}(w)$. Now we could obtain w by calling $\text{Decompress}(\text{Compress}(w))$.

Intuitively, the output $Z + |\text{Compress}(w)|$ preserves ϵ -DP since this is just the standard Laplacian Mechanism and the output $\lceil Z + |\text{Compress}(w)| + k \rceil = |\text{Compress}(w)| + \lceil Z + k \rceil$ also preserves ϵ -DP since it can be viewed as post-processing applied to a DP output. Finally, note that by our choice of k we have $\Pr[p \neq \lceil Z + k \rceil] = \Pr[Z + k \leq 0] \leq \delta$. It follows that the output $|\text{Compress}(w)| + p$ preserves (ϵ, δ) -DP, as shown in [Theorem 2](#).

Algorithm 1 $\text{DPCompress}(w, \epsilon, \delta)$

Input: w, ϵ, δ

Output: Differentially Private Padded Data.

$Z \leftarrow \text{Lap}\left(\frac{\text{GS}_{\text{Compress}}}{\epsilon}\right)$

$k \leftarrow \frac{\text{GS}_{\text{Compress}}}{\epsilon} \ln\left(\frac{1}{2\delta}\right) + \text{GS}_{\text{Compress}} + 1$

$p \leftarrow \max\{1, \lceil Z + k \rceil\}$

Return $\text{pad}(\text{Compress}(w), p) := \text{Compress}(w) \circ 0 \circ 1^{p-1}$

Theorem 2. Define $A_{\epsilon, \delta}(w) := \text{DPCompress}(w, \epsilon, \delta)$ then, for any $\epsilon, \delta > 0$, the algorithm $A_{\epsilon, \delta}$ is a (ϵ, δ) -differentially private compression scheme.

The proof of [Theorem 2](#) is straightforward using standard DP techniques and therefore is deferred to [Appendix A](#).

On average, the amount of padding added is approximately $k = \frac{\text{GS}_{\text{Compress}}}{\epsilon} \ln\left(\frac{1}{2\delta}\right) + \text{GS}_{\text{Compress}} + 1$. If $k = o(n)$ then it will still be possible for DPCompress to achieve efficient compression ratios, i.e., $|\text{DPCompress}(w, \epsilon, \delta)|/n = |\text{Compress}(w)|/n + o(1)$. On the other hand, if $k = \Omega(n)$ then the padding alone will prevent us from achieving a compression ratio of $o(1)$. Thus, our hope is to find practical compression schemes with global sensitivity $\text{GS}_{\text{Compress}} = o(n)$ — LZ78 [17] does not satisfy these criteria [15]. This motivates our study of the global sensitivity of the LZ77 compression scheme [16] — see [Section 4](#) for details.

4 Upper Bound for the Global Sensitivity of LZ77

Recall that in [Section 3](#), we provided the framework to convert any compression scheme to a (ϵ, δ) -DP compression scheme by adding a random amount of padding $p = \max\{1, \lceil Z + k \rceil\}$, where $Z \sim \text{Lap}(\text{GS}_{\text{Compress}}/\epsilon)$ and $k = \frac{\text{GS}_{\text{Compress}}}{\epsilon} \ln\left(\frac{1}{2\delta}\right) + \text{GS}_{\text{Compress}} + 1$ is a constant. We observed that as long as $k = o(n)$, we can still argue that DPCompress achieves efficient compression ratios, i.e., $|\text{DPCompress}(w, \epsilon, \delta)|/n = |\text{Compress}(w)|/n + o(1)$. That was the motivation to find practical compression schemes with global sensitivity $o(n)$. We argue that the LZ77 compression

scheme [16] satisfies this property. For simplicity of exposition, we will assume that $W = n$ in most of our analysis and then briefly explain how the analysis changes when $W < n$. In particular, we prove that the global sensitivity of the LZ77 compression scheme is $\mathcal{O}(W^{2/3} \log n)$ or $\mathcal{O}(n^{2/3} \log n)$ when $W = n$.

4.1 Analyzing the Positions of Blocks

Recall that LZ77 [16] with the compression function $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ outputs a sequence of blocks $B_1, \dots, B_t$ where each block is of the form $B_i = [q_i, \ell_i, c_i]$ such that $0 \leq q_i, \ell_i < n$ are nonnegative integers and $c_i \in \Sigma$ is a character. This implies that for a string $w \in \Sigma^n$, it takes $2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil$ bits to encode each block and this is the same for all the blocks. Therefore, the length of compression is proportional to the number of blocks t , i.e., we have $|\text{Compress}(w)| = t(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$. Let $B'_1, \dots, B'_{t'}$ denotes the blocks when compressing w' instead of w . Our observation above tells us that to analyze the global sensitivity of the LZ77 compression scheme, it is crucial to understand the upper/lower bound of $t' - t$ (WLOG we can assume $t' \geq t$ since we can always change the role of w and w') where t (resp. t') is the number of blocks in $\text{Compress}(w)$ (resp. $\text{Compress}(w')$) for neighboring strings $w \sim w' \in \Sigma^n$.

To analyze the difference between the number of blocks $t' - t$, it is helpful to introduce some notation. First, since $w \sim w'$ we will use $j \leq n$ to denote the unique index such that $w[j] \neq w'[j]$ — note that $w[i] = w'[i]$ for all $i \neq j$. Second, the block $B_i = [q_i, \ell_i, c_i]$ can be viewed intuitively as an instruction for the decompression algorithm to locate the substring $w[q_i, q_i + \ell_i - 1]$ from the part of w that we have already decompressed, copy this substring and append it to the end of the of the decompressed file followed by the character c_i . While the inputs q_i and ℓ_i tell us where to copy *from* it is also useful to let $s_i := 1 + \sum_{r=1}^{i-1} (\ell_r + 1)$ and $f_i := \sum_{r=1}^i (\ell_r + 1)$ denote the location where the block is copied *to*, i.e., we have $w[s_i, f_i] = w[q_i, q_i + \ell_i - 1] \circ c_i$.

We say that block B'_k starts inside block B_i if $s_i \leq s'_k \leq f_i$ and we indicate this with the predicate $\text{StartInside}(i, k) := 1$. Otherwise, if $s'_k < s_i$ or $s'_k > f_i$ then we have $\text{StartInside}(i, k) := 0$. Our key technical insight is that if $\text{StartInside}(i, k) = 1$ then for block B'_{k+1} we must have $f'_{k+1} \geq f_k$. In particular, for later blocks $B'_{k'}$ with $k' > k$ we will have $s'_{k'} > f_i$ so $\text{StartInside}(i, k') = 0$. In particular, if we let $\mathcal{M}_i := \{k \in [t'] : \text{StartInside}(i, k) = 1\}$ then Lemma 1 tells us that one of three cases applies: (1) $\mathcal{M}_i = \emptyset$, (2) $\mathcal{M}_i = \{k\}$ for some $k \leq t'$, or (3) $\mathcal{M}_i = \{k, k+1\}$ for some $k < t'$. In any case, we have $|\mathcal{M}_i| \leq 2$. This has already been proven by Akagi et al. [1], but for the sake of completeness, we include our proof in Appendix B.1.

Lemma 1 ([1]). *Let $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression algorithm and $w, w' \in \Sigma^n$ such that $w \sim w'$. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Then for all $i \in [t]$, either $\mathcal{M}_i = \emptyset$ or $\mathcal{M}_i = [i_1, i_2]$ for some $i_1 \leq i_2 \leq i_1 + 1$. In particular, $|\mathcal{M}_i| \leq 2, \forall i \in [t]$.*

Now we can partition the blocks $B_1, \dots, B_t$ into three sets based on the size of $\mathcal{M}_i$. In particular, let $\mathcal{B}_m := \{i : |\mathcal{M}_i| = m\}$ for $m \in \{0, 1, 2\}$ — Lemma 1

implies that $\mathcal{B}_m = \emptyset$ for $m \geq 3$. To upper bound $t' - t$, it is essential to *count the number of type-2 blocks*. Let $t_m = |\mathcal{B}_m|$ be the number of type- m blocks for $m = 0, 1, 2$. Then we have the following claim.

Claim 1. *Let $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression function and w, w' be strings of length n and $w \sim w'$. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Then $t' - t \leq t_2$.*

Proof. We observe $t_0 + t_1 + t_2 = t$ by Lemma 1 and since there are m blocks in $(B'_1, \dots, B'_{t'})$ that start inside type- m blocks in $(B_1, \dots, B_t)$ for $m = 0, 1, 2$, we have $t' = \sum_{m=0}^2 m \cdot t_m = t_1 + 2t_2$, which implies that $t' - t = t_2 - t_0 \leq t_2$. $\square$

Claim 1 implies that $|\text{Compress}(w)| - |\text{Compress}(w')| \leq (t' - t)(2 \lceil \log n \rceil + \lceil \log |\Sigma| \rceil) \leq t_2(2 \lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$ since it takes $2 \lceil \log n \rceil + \lceil \log |\Sigma| \rceil$ bits to encode each block (see Claim 15 in Appendix B.1). Hence, upper bounding the number of type-2 blocks t_2 would allow us to upper bound the global sensitivity of the LZ77 compression.

4.2 Counting Type-2 Blocks using Uniqueness of Offsets

We can effectively count the number of type-2 blocks by considering the location of the unique index j such that $w[j] \neq w'[j]$ and show that the number of type-2 blocks is at most $\mathcal{O}(n^{2/3})$ as stated in Lemma 2.

Lemma 2. *Let $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression function and w, w' be strings of length n and $w \sim w'$. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Then $t_2 \leq \frac{\sqrt[3]{9}}{2} n^{2/3} + \frac{\sqrt[3]{3}}{2} n^{1/3} + 1$.*

Proof Sketch. We only give the proof sketch here and the formal proof can be found in Appendix B.2. To prove Lemma 2, we show the following helper claims.

- First, we show that if B_i is a type-2 block then either $s_i \leq j \leq f_i$ or $q_i \leq j < q_i + \ell_i$ should hold (see Claim 16 in Appendix B.2). Intuitively, any type-2 block B_i with $s_i > j$ is constructed by copying the longest possible substring *from* a section around the index j i.e., $j \in [q_i, q_i + \ell_i)$. Type-2 blocks cannot occur before the strings diverge at index j i.e., if $f_i < j$ then B_i is not a type-2 block.
- Second, we show that if blocks $B_{i_1} = [q_{i_1}, \ell_{i_1}, c_{i_1}]$ and $B_{i_2} = [q_{i_2}, \ell_{i_2}, c_{i_2}]$ are both type-2 blocks with $s_{i_1}, s_{i_2} > j$ then $(q_{i_1}, \ell_{i_1}) \neq (q_{i_2}, \ell_{i_2})$ (see Claim 17 in Appendix B.2). In particular, the pair (q_i, ℓ_i) must be *unique* for each type-2 block and, if $s_i > j$, then we must have $j \in [q_i, q_i + \ell_i)$ by our observation above.
- Finally, followed by the uniqueness of offsets from the previous claim, we can show that the number of type-2 blocks with length ℓ is at most ℓ by pigeonhole principle, i.e., if $\mathcal{B}_2^\ell := \{i \in \mathcal{B}_2 : \ell_i = \ell\}$ and if $s_{i^*} \leq j \leq f_{i^*}$ for some $i^* \in [t]$, then $|\mathcal{B}_2^\ell| \leq \ell$ for all $\ell \neq \ell_{i^*}$ and $|\mathcal{B}_2^{\ell_{i^*}}| \leq \ell_{i^*} + 1$ (see Claim 18 in Appendix B.2).

Let $x_\ell := |\mathcal{B}_2^\ell|$ denote the number of type-2 blocks with length ℓ . The total number of type-2 blocks is given by the sum $\sum_\ell x_\ell$. If we try to maximize $\sum_\ell x_\ell$ subject to the constraints that $x_\ell \leq \ell$ (for all $\ell \neq \ell_{i^*}$), $x_{\ell_{i^*}} \leq \ell_{i^*} + 1$, and $\sum_\ell x_\ell(\ell + 1) \leq n$, we obtain a solution where we set $x_\ell = \ell$ for $\ell \leq z$ with $\ell \neq \ell_{i^*}$ (and set $x_{\ell_{i^*}} = \ell_{i^*} + 1$ if $\ell_{i^*} \leq z$) and $x_\ell = 0$ for $\ell > z$ by a simple swapping argument (see the formal proof in [Appendix B.2](#) for details). Note that x_z could be less than z , i.e., for the threshold z , we have $x_z = c$ for some $0 < c \leq z$. To find the threshold z , we observe that $\left[\sum_{\ell=0}^{z-1} \ell(\ell + 1) \right] + x_z(z + 1) = \sum_{\ell \geq 0} x_\ell(\ell + 1) \leq n$. Simplifying the sum, we have $\left[\sum_{\ell=0}^{z-1} \ell(\ell + 1) \right] + x_z(z + 1) = \frac{1}{3}(z-1)z(z+1) + x_z(z+1) \geq z^3/3$ where the last inequality follows because $x_z \geq 1$. Thus, we have $z^3 \leq 3n$ and $z \leq \sqrt[3]{3n}$. Setting this value of z , we have $t_2 \leq \left(\sum_{\ell=0}^{\sqrt[3]{3n}} \ell \right) + 1 = \frac{\sqrt[3]{9}}{2}n^{2/3} + \frac{\sqrt[3]{3}}{2}n^{1/3} + 1$. $\square$

While we focus on analyzing the global sensitivity of the *non-overlapping* LZ77 compression scheme, one can easily extend the results to the LZ77 with self-referencing and achieve the same upper bound for $t_2 = \mathcal{O}(n^{2/3})$. See [Appendix B.3](#) for details.

We remark that this is a key result to upper bound the global sensitivity of the LZ77 compression scheme from our previous discussion that it takes $\mathcal{O}(\log n)$ bits to encode each block. Since the number of type-2 blocks is $\mathcal{O}(n^{2/3})$ and $t' - t$ is bounded by the number of type-2 blocks, we can combine those results and conclude that the global sensitivity of the LZ77 compression scheme is upper bounded by $\mathcal{O}(n^{2/3} \log n)$, as stated in [Theorem 3](#).

Theorem 3. *Let $\text{Compress} : (\Sigma)^* \rightarrow (\Sigma')^*$ be the LZ77 compression function with unbounded sliding window size $W = n$. Then*

$$\text{GS}_{\text{Compress}} \leq \left(\frac{\sqrt[3]{9}}{2}n^{2/3} + \frac{\sqrt[3]{3}}{2}n^{1/3} + 1 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) = \mathcal{O}(n^{2/3} \log n).$$

Proof. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. By [Claim 1](#) and [Lemma 2](#), we have $t' - t \leq t_2 \leq \frac{\sqrt[3]{9}}{2}n^{2/3} + \frac{\sqrt[3]{3}}{2}n^{1/3} + 1$. We know $|\text{Compress}(w)| = t(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$ and $|\text{Compress}(w')| = t'(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$. Hence, $||\text{Compress}(w)| - |\text{Compress}(w')|| = |t - t'| (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) \leq \left(\frac{\sqrt[3]{9}}{2}n^{2/3} + \frac{\sqrt[3]{3}}{2}n^{1/3} + 1 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$. Since this inequality hold for arbitrary $w \sim w'$ of length n , we have

$$\begin{aligned} \text{GS}_{\text{Compress}} &= \max_{w \in \Sigma^n} \max_{w' \in \Sigma^n: w \sim w'} ||\text{Compress}(w)| - |\text{Compress}(w')|| \\ &\leq \left(\frac{\sqrt[3]{9}}{2}n^{2/3} + \frac{\sqrt[3]{3}}{2}n^{1/3} + 1 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) = \mathcal{O}(n^{2/3} \log n). \quad \square \end{aligned}$$

For the LZ77 with self-referencing, [Lemma 1](#) essentially still holds except that we might have one type-3 block (i.e., $t_3 = 1$), i.e., the block containing the

edited position j — see [1, p.18]. Thus, the global sensitivity of the LZ77 with self-referencing would be bounded by $\mathcal{O}(t_2 \log n + 2t_3 \log n) = \mathcal{O}(t_2 \log n + 2 \log n) = \mathcal{O}(n^{2/3} \log n)$. See [Theorem 6](#) in [Appendix B.3](#) for details.

4.3 Bounded Sliding Window ($W < n$)

The analysis for the case when $W < n$ is very similar. When $W < n$ then we obtain an additional constraint on any type-2 block as stated in [Claim 2](#).

Claim 2. *Let $\text{Compress} : (\Sigma)^* \rightarrow (\Sigma')^*$ be the LZ77 compression function with sliding window size W and $w \sim w'$ be strings of length n with $w[j] \neq w'[j]$. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. If $B_i \in \mathcal{B}_2$ then $s_i \leq j + W$.*

The proof of [Claim 2](#) is elementary and can be found in [Appendix B.4](#). [Claim 2](#) tells us that there is no type-2 block that starts after $j + W$. Prior constraints still apply: we still have $f_i \geq j$ for type-2 blocks B_i and if $s_i \geq j$ then we must have $j \in [q_i, q_i + \ell_i]$. In particular, we can have at most ℓ type-2 blocks of length ℓ and, if $s_i \geq j$, we must have $j \in [q_i, q_i + \ell_i]$. Letting x_ℓ denote the number of type-2 blocks of length ℓ (excluding any block B_i such that $j \in [s_i, f_i]$ or such that $f_i > j + W$ if any such type-2 blocks exist⁶) we now obtain the constraint that $\sum_\ell (\ell + 1)x_\ell \leq 3W$ instead of the prior constraint that $\sum_\ell (\ell + 1)x_\ell \leq n$. Maximizing $\sum_\ell x_\ell$ subject to the above constraint, as well as the prior constraints that $x_\ell \leq \ell$ for all ℓ , we can now obtain a tighter upper bound $t_2 = \mathcal{O}(W^{2/3})$ instead of $t_2 = \mathcal{O}(n^{2/3})$. This leads to the following result.

Theorem 4. *Let $\text{Compress} : (\Sigma)^* \rightarrow (\Sigma')^*$ be the LZ77 compression function with sliding window size W . Then $\text{GS}_{\text{Compress}} \leq \left(\frac{\sqrt[3]{81}}{2} W^{2/3} + \frac{\sqrt[3]{9}}{2} W^{1/3} + 3 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) = \mathcal{O}(W^{2/3} \log n)$.*

Essentially the same result holds for the LZ77 with self-referencing as well; see [Theorem 7](#) in [Appendix B.4](#) for details.

In practice, many implementations of LZ77 set $W \ll n$ to minimize space usage. In addition to minimizing space usage, these implementations also reduce the global sensitivity of the compression algorithm.

5 Lower Bound for the Global Sensitivity of LZ77

In [Section 4](#), we proved that the upper bound for the global sensitivity of the LZ77 compression algorithm **Compress** is $\mathcal{O}(n^{2/3} \log n)$ with window size $W = n$. One could ask if this is a tight bound, i.e., if we can prove the matching *lower bound* for the global sensitivity of **Compress** as well. This section

⁶ Observe that we exclude *at most* two blocks since there can be *at most* one type-2 block B_i with $f_i > j + W$ and there can be *at most* one type two block B_i with $j \in [s_i, f_i]$.

proves the almost-matching lower bound up to a sub-logarithmic factor. In particular, we show that the global sensitivity of the LZ77 compression algorithm is $\Omega(n^{2/3} \log^{1/3} n)$. To prove the lower bound, we need to give example strings $w \sim w'$ of length n that achieves $|\text{Compress}(w)| - |\text{Compress}(w')| = \Omega(n^{2/3} \log^{1/3} n)$, since this implies $\text{GS}_{\text{Compress}} = \max_{x \in \Sigma^n} \max_{x' \in \Sigma^n : x \sim x'} |\text{Compress}(x)| - |\text{Compress}(x')| \geq |\text{Compress}(w)| - |\text{Compress}(w')| = \Omega(n^{2/3} \log^{1/3} n)$. In this section, we will give the construction of such example strings w and w' .

5.1 String Construction

Consider an encoding function $E : \mathbb{Z} \rightarrow \{0, 1\}^*$ that maps integers to binary strings. Then for a positive integer $m \in \mathbb{Z}$, we have an injective encoding of the number set $\mathcal{S} := \{0, 1, \dots, m\}$ using $\lceil \log m \rceil$ bits, i.e., $E(i) \neq E(j)$ if $i, j \in \mathcal{S}$ and $i \neq j$. For example, if $m = 2^q - 1$ for some positive integer q , we could encode the elements of $\mathcal{S}$ as follows:

$$E(0) = 0^{\lceil \log m \rceil}, E(1) = 0^{\lceil \log m \rceil - 1} 1, E(2) = 0^{\lceil \log m \rceil - 2} 10, \dots, E(m) = 1^{\lceil \log m \rceil}.$$

Now, consider a quinary alphabet $\Sigma = \{0, 1, 2, 3, 4\}$ and define a string

$$S_{\ell, u} := E(m - u + 1)^2 \circ E(m - u + 2)^2 \circ \dots \circ E(m)^2 \circ 2 \circ E(m + 1)^2 \circ \dots \circ E(m - u + \ell)^2$$

in Σ^* for $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$. Here, $(\cdot)^2$ denotes the concatenation of the string itself twice, i.e., $E(\cdot)^2 = E(\cdot) \circ E(\cdot)$. We define a procedure called $\text{QuinStr}(m)$ which takes as input a positive integer $m \in \mathbb{Z}$ and outputs two quinary strings as follows.

The Construction of Two Quinary Strings $\text{QuinStr}(m)$.

1. Compute two quinary strings S_w and $S_{w'}$ where

$$S_w := E(1)^2 \circ \dots \circ E(m)^2 \circ 2 \circ E(m + 1)^2 \circ \dots \circ E(2m)^2 \circ 4, \text{ and} \\ S_{w'} := E(1)^2 \circ \dots \circ E(m)^2 \circ 3 \circ E(m + 1)^2 \circ \dots \circ E(2m)^2 \circ 4.$$

2. Compute two quinary strings $w := S_w \circ S$ and $w' := S_{w'} \circ S$, where

$$S = S_{2,1} \circ 4 \circ S_{3,2} \circ 4 \circ S_{3,1} \circ 4 \circ \dots \circ S_{m,m-1} \circ 4 \circ \dots \circ S_{m,1} \circ 4.$$

3. Output (w, w') .

Claim 3 tells us that the strings w and w' outputted by the procedure $\text{QuinStr}(m)$ has equal length $\Theta(m^3 \log m)$. Since the proof is elementary, we defer the proof of **Claim 3** to **Appendix C**.

Claim 3. *Let $m \in \mathbb{N}$ and $(w, w') \leftarrow \text{QuinStr}(m)$. Then $|w| = |w'| = \Theta(m^3 \log m)$. In particular, for $m \geq 4$, $\frac{2}{3}m^3 \lceil \log m \rceil < |w| = |w'| < m^3 \lceil \log m \rceil$.*

5.2 Analyzing the Sensitivity of QuinStr(m)

A central step in our sensitivity analysis for QuinStr(m) is precisely counting the type-2 blocks produced by the LZ77 compression scheme, as we did in Section 4. Lemma 3 shows that for $(w, w') \leftarrow \text{QuinStr}(m)$, we have $|\mathcal{B}_2| = \frac{(m-1)m}{2} - (\lfloor \frac{m}{2} \rfloor - 1)$. Intuitively, we first show that for $w = S_w \circ S$, there is no type-2 block for the blocks compressing S_w . Then the main insight is that we carefully crafted strings w and w' such that the marker symbol '4' becomes the endpoint for each block in $\text{Compress}(w)$ for the tail part S of $w = S_w \circ S$. By repeating each encoding twice, we can ensure that most of the occurrences of $S_{\ell,u} \circ 4$ yield type-2 blocks, with an edge case (addressed in Claim 7) that makes the block in $\mathcal{B}_1$ but this happens for only about $m/2$ blocks. Consequently, despite these few exceptions, the overall count of type-2 blocks remains quadratic in m .

Lemma 3. *Let $m \in \mathbb{N}$ and $(w, w') \leftarrow \text{QuinStr}(m)$ and let $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression algorithm. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Then $|\mathcal{B}_0| = 0$ and $|\mathcal{B}_2| = \frac{(m-1)m}{2} - (\lfloor \frac{m}{2} \rfloor - 1)$.*

Proof. Recall that $w = S_w \circ S$ and $w' = S_{w'} \circ S$, where

- $S_w = \text{E}(1)^2 \circ \dots \circ \text{E}(m)^2 \circ 2 \circ \text{E}(m+1)^2 \circ \dots \circ \text{E}(2m)^2 \circ 4$,
- $S_{w'} = \text{E}(1)^2 \circ \dots \circ \text{E}(m)^2 \circ 3 \circ \text{E}(m+1)^2 \circ \dots \circ \text{E}(2m)^2 \circ 4$, and
- $S = S_{2,1} \circ 4 \circ S_{3,2} \circ 4 \circ S_{3,1} \circ 4 \circ \dots \circ S_{m,m-1} \circ 4 \circ \dots \circ S_{m,1} \circ 4$, where
- $S_{\ell,u} := \text{E}(m-u+1)^2 \circ \text{E}(m-u+2)^2 \circ \dots \circ \text{E}(m)^2 \circ 2 \circ \text{E}(m+1)^2 \circ \dots \circ \text{E}(m-u+\ell)^2$ for $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$.

We first observe that $w \sim w'$. Define $S_w^F := \text{E}(1)^2 \circ \dots \circ \text{E}(m)^2 \circ 2$ (resp. $S_{w'}^F := \text{E}(1)^2 \circ \dots \circ \text{E}(m)^2 \circ 3$) to be the first-half substring of S_w (resp. of $S_{w'}$), and $S_w^L = S_{w'}^L := \text{E}(m+1)^2 \circ \dots \circ \text{E}(2m)^2 \circ 4$ to be the last-half substring of S_w (or $S_{w'}$ since they are indeed identical). It is useful to define a notation $\text{str}(B_k)$ for a block B_k , which denotes the substring of w represented by the block B_k , i.e., for $B_k = [q_k, \ell_k, c_k]$, $\text{str}(B_k) := w[q_k, q_k + \ell_k - 1] \circ c_k$.

Let B_{i_1} be the first block such that S_w^F becomes a substring of $\text{str}(B_1) \circ \text{str}(B_2) \circ \dots \circ \text{str}(B_{i_1})$, and similarly, let $B'_{i'_1}$ be the first block such that $S_{w'}^F$ becomes a substring of $\text{str}(B'_1) \circ \text{str}(B'_2) \circ \dots \circ \text{str}(B'_{i'_1})$. Then we observe the following:

- (1) $B_{i_1} = [q_{i_1}, \ell_{i_1}, 2]$, i.e., $\text{str}(B_{i_1})$ ends with 2 (which is the last character in S_w^F), since 2 never showed up before as all the encodings are binary strings, it has to be added to the dictionary as a new character,
- (2) $B_i = B'_i$ for all $i \in [i_1 - 1]$, as we are compressing the identical strings until we see 2 in S_w^F (and 3 in $S_{w'}^F$), and
- (3) $i_1 = i'_1$ and $B_{i_1} = B'_{i_1} = [q_{i_1}, \ell_{i_1}, 3]$, since two strings S_w^F and $S_{w'}^F$ are identical except for the very last character.

Now let B_{i_2} be the first block such that S_w^L becomes a substring of $\text{str}(B_{i_1+1}) \circ \text{str}(B_{i_1+2}) \circ \dots \circ \text{str}(B_{i_2})$, and similarly, let $B'_{i'_2}$ be the first block such that $S_{w'}^L$ becomes a substring of $\text{str}(B'_{i_1+1}) \circ \text{str}(B'_{i_1+2}) \circ \dots \circ \text{str}(B'_{i'_2})$. Then we observe the following:

- (4) $i_2 = i'_2$ and $B_i = B'_i$ for all $i \in [i_1 + 1, i_2]$, since $i_1 = i'_1$ from observation 3 and we have $S_w^L = S_{w'}^L$, while they do not contain 2 or 3, and
- (5) $B_{i_2} = [q_{i_2}, \ell_{i_2}, 4]$, since 4 never showed up before in our compression.

From the observations above, we have that $B_i \in \mathcal{B}_1$ for all $i \in [i_2]$. Now we are left with the blocks $(B_{i_2+1}, \dots, B_t)$ compressing the last part S of w and the blocks $(B'_{i_2+1}, \dots, B'_t)$ compressing the last part S of w' . For the blocks $(B_{i_2+1}, \dots, B_t)$, we observe that each block ends at the next '4' because each $S_{\ell,u}$ (for $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$) is contained in the former part of w (which was S_w) but 4 only shows up in S_w followed by $E(2m)^2$ while $S_{\ell,u}$ cannot contain $E(2m)$. Hence, we observe the following:

- (6) $\text{str}(B_{i_2+1}) = S_{2,1} \circ 4$, $\text{str}(B_{i_2+2}) = S_{3,2} \circ 4$, and so on. To see this, we recall that $S_{\ell,u} := E(m-u+1)^2 \circ E(m-u+2)^2 \circ \dots \circ E(m)^2 \circ 2 \circ E(m+1)^2 \circ \dots \circ E(m-u+\ell)^2$. Because of '2' in the middle of each $S_{\ell,u}$ and '4' at the end of each $S_{\ell,u} \circ 4$, we observe that $S_{\ell,u} \circ 4$ cannot be copied from a previous position in S .
- (7) In general, $\text{str}\left(B_{i_2 + \frac{(\ell-2)(\ell-1)}{2} + (\ell-u)}\right) = S_{\ell,u} \circ 4$, for $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$. Intuitively, the blocks are numbered by iterating over $\ell = 2, \dots, m$, and for each ℓ , iterating over $u = \ell - 1, \dots, 1$ in reverse order. This indeed covers all the blocks from $B_{i_2+1}, \dots, B_t$ (See Claim 4 and observation 8).
- (8) Furthermore, we can observe that $t = i_2 + (1+2+\dots+(m-1)) = i_2 + \frac{(m-1)m}{2}$.

Claim 4. For any integer $m \geq 2$, the function $f(\ell, u) := \frac{(\ell-2)(\ell-1)}{2} + (\ell - u)$ defined over integers ℓ and u such that $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$ is injective, and its range is $[\frac{(m-1)m}{2}]$.

The proof of Claim 4 is elementary by induction on m , and hence, we defer the proof to Appendix C. What we are interested in is whether each B_i , for $i_2 + 1 \leq i \leq t$, belongs to $\mathcal{B}_0$, $\mathcal{B}_1$, or $\mathcal{B}_2$. In Claim 5, we prove that the blocks are mostly in $\mathcal{B}_2$ and the rest of the blocks are in $\mathcal{B}_1$, meaning that $\mathcal{B}_0 = \emptyset$. In particular, we prove that for $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$, $B_{i_2 + \frac{(\ell-2)(\ell-1)}{2} + (\ell-u)} \in \mathcal{B}_1$ if and only if all of these conditions hold: (1) $\ell > 2$, (2) ℓ is even, and (3) $u = \ell/2$.

Claim 5. For $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$,

- $B_{i_2 + \frac{(\ell-2)(\ell-1)}{2} + (\ell-u)} \in \mathcal{B}_1$ if and only if $\ell > 2$, $2 \mid \ell$, and $u = \ell/2$;
- Otherwise $B_{i_2 + \frac{(\ell-2)(\ell-1)}{2} + (\ell-u)} \in \mathcal{B}_2$.

We will give the proof of Claim 5 below and finish the proof of Lemma 3 first for readability. By Claim 5, since there are only $\lfloor \frac{m}{2} \rfloor - 1$ of such pairs of (ℓ, u) (satisfying $\ell > 2$, $2 \mid \ell$, and $u = \ell/2$), we observe that $|\mathcal{B}_2| = \frac{(m-1)m}{2} - (\lfloor \frac{m}{2} \rfloor - 1)$. Since we have that $B_i \in \mathcal{B}_1$ for all $i \in [i_2]$, we have $\mathcal{B}_0 = \emptyset$ and therefore $|\mathcal{B}_0| = 0$. This completes the proof of Lemma 3. $\square$

Proof of Claim 5. Recall that $S = S_{2,1} \circ 4 \circ S_{3,2} \circ 4 \circ S_{3,1} \circ 4 \circ \dots \circ S_{m,m-1} \circ 4 \circ \dots \circ S_{m,1} \circ 4$ and $\text{str}(B_{i_2+1}) = S_{2,1} \circ 4$, $\text{str}(B_{i_2+2}) = S_{3,2} \circ 4$, $\text{str}(B_{i_2+3}) =$

$S_{3,1} \circ 4, \dots, \text{str}(B_t) = S_{m,1} \circ 4$. For each $S_{\ell,u}$, we observe that $S_{\ell,u}$ is *not* a substring of $S_{w'}$. Hence, we see that each block B_i (for $i_2 + 1 \leq i \leq t$) is roughly split into two blocks for the blocks of $\text{Compress}(w')$ unless it could copy beyond the character 4. To observe the cases when this happens, for each $S_{\ell,u}$, it is helpful to define:

- $S_{\ell,u}^F := E(m - u + 1)^2$ denotes the very first encoding concatenation that shows in $S_{\ell,u}$,
- $S_{\ell,u}^{F,(1/2)} := E(m - u + 1)$ denotes the very first encoding in $S_{\ell,u}$ (i.e., half of $S_{\ell,u}^F$),
- $S_{\ell,u}^L := E(m - u + \ell)^2$ denotes the very last encoding concatenation that shows in $S_{\ell,u}$, and
- For $k > u$, $S_{\ell,u}^{(k)} := E(m - u + 1)^2 \circ E(m - u + 2)^2 \circ \dots \circ E(m)^2 \circ 2 \circ E(m + 1)^2 \circ \dots \circ E(m - u + k)^2$ denotes the first k encoding concatenations that shows in $S_{\ell,u}$.

Then we observe the following claims. Since proofs of [Claim 6](#) and [Claim 7](#) are elementary, we defer the proofs to [Appendix C](#).

Claim 6. $S_{\ell,u}^L \circ 4 \circ S_{\ell,u-1}^F$ does not repeat for different ℓ and u such that $3 \leq \ell \leq m$ and $2 \leq u \leq \ell - 1$.

[Claim 6](#) tells us that, due to the injectivity of the encoding, any block in $\text{Compress}(w')$ containing a portion of $S_{\ell,u}^L$ along with the delimiter ‘4’ must finish at $S_{\ell,u}^{F,(1/2)}$ in the worst case. In particular, note that $S_{\ell,u}^L = E(m - u + \ell)^2 = S_{\ell+1,u+1}^L$ for $3 \leq \ell < m$ and $2 \leq u < \ell - 1$. Moreover, we have $S_{\ell,u-1}^{F,(1/2)} = E(m - u + 2)$ and $S_{\ell+1,u}^{F,(1/2)} = E(m - u + 1)$, which can agree on all but the final bit (e.g., $S_{\ell,u-1}^{F,(1/2)} = 00 \dots 00$ and $S_{\ell+1,u}^{F,(1/2)} = 00 \dots 01$). Without the repetition of each encoding, a block might incorporate nearly the entire $S_{\ell+1,u}^{F,(1/2)}$ except for the last bit. Consequently, by having this last bit as a new character, $S_{\ell,u-1} \circ 4$ would be placed in $\mathcal{B}_1$. Repeating the encoding twice eliminates this possibility and we can ensure that the scenario described in [Claim 7](#) is the only case where type-1 blocks would occur. Again, see [Appendix C](#) for the proof of [Claim 7](#).

Claim 7. For $2 \leq \ell \leq \lfloor \frac{m}{2} \rfloor - 1$, $S_{\ell,1}^L \circ 4 \circ S_{\ell+1,\ell}$ repeats at $S_{2\ell,\ell+1}^L \circ 4 \circ S_{2\ell,\ell}^{(\ell+1)}$.

Let’s go back to the proof of [Claim 5](#). By [Claim 7](#), we can see that the block of the form $B_{i_2 + \frac{(2\ell-2)(2\ell-1)}{2} + (2\ell-\ell)}$ which satisfies

$$\text{str}\left(B_{i_2 + \frac{(2\ell-2)(2\ell-1)}{2} + (2\ell-\ell)}\right) = S_{2\ell,\ell} \circ 4,$$

is in $\mathcal{B}_1$, and all of the other blocks beyond B_{i_2} are in $\mathcal{B}_2$. This completes the proof of [Claim 5](#). $\square$

Taken altogether, we can lower bound the global sensitivity of the LZ77 compression scheme as stated in [Theorem 5](#) below.

Theorem 5. *Let $\text{Compress} : \Sigma^* \rightarrow \Sigma'^*$ be the LZ77 compression function. Then $\text{GS}_{\text{Compress}} \geq 4^{-1/3} \cdot n^{2/3} \log^{1/3} n = \Omega(n^{2/3} \log^{1/3} n)$.*

Proof. Let $(w, w') \leftarrow \text{QuinStr}(m)$ and let $|w| = |w'| = n$. By [Claim 3](#), we have $|w| = |w'| = \Theta(m^3 \log m)$ and therefore $n = \Theta(m^3 \log m)$. Furthermore, [Claim 3](#) tells us that there exists some α with $\frac{2}{3} \leq \alpha \leq 1$ such that $n = \alpha m^3 \log m$. Now let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Recall that if we look at the proof of [Claim 1](#), it tells us that $t' - t = |\mathcal{B}_2| - |\mathcal{B}_0|$. From [Lemma 3](#), we have $|\mathcal{B}_0| = 0$ and $|\mathcal{B}_2| = \frac{(m-1)m}{2} - (\lfloor \frac{m}{2} \rfloor - 1)$, which implies that $t' - t = \frac{(m-1)m}{2} - (\lfloor \frac{m}{2} \rfloor - 1)$. We know $|\text{Compress}(w)| = t(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$ and $|\text{Compress}(w')| = t'(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$, we have

$$\begin{aligned} \text{GS}_{\text{Compress}} &\geq ||\text{Compress}(w)| - |\text{Compress}(w')|| \\ &= |t - t'| (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) \\ &= |t - t'| (2\lceil \log(\alpha m^3 \log m) \rceil + \lceil \log |\Sigma| \rceil) \\ &\geq \frac{m^2}{4} \cdot 4 \log m = m^2 \log m. \end{aligned}$$

Furthermore, since we have $n = \alpha m^3 \log m$ for some $\frac{2}{3} \leq \alpha \leq 1$, we observe that

$$\begin{aligned} m^2 \log m &= m^2 \cdot \frac{n}{\alpha m^3} = \frac{n}{\alpha} \cdot \frac{1}{m} = \frac{n}{\alpha} \cdot \frac{\alpha^{1/3} \cdot \log^{1/3} m}{n^{1/3}} \\ &\geq \left(\frac{n}{\alpha}\right)^{2/3} \cdot 4^{-1/3} \cdot \log^{1/3} n \\ &\geq 4^{-1/3} \cdot n^{2/3} \log^{1/3} n, \end{aligned}$$

where the first inequality comes from the observation $\log n = \log \alpha + 3 \log m + \log \log m \leq 4 \log m$ and the second inequality comes from $(1/\alpha) \geq 1$. Hence,

$$\text{GS}_{\text{Compress}} \geq m^2 \log m \geq 4^{-1/3} \cdot n^{2/3} \log^{1/3} n,$$

which completes the proof. $\square$

6 Is High Sensitivity Inevitable?

Based on the results of Lagarde and Perifel [\[15\]](#), one might wonder whether or not any effective compression mechanism will necessarily have *high* global sensitivity. We argue that this is not necessarily the case by considering *Kolmogorov compression*. The Kolmogorov compression of an input string w is simply the encoding of the minimum-size Turing Machine M such that the Turing Machine M will eventually output w when run with an initially empty input tape. It is also easy to see that the Kolmogorov compression scheme has low global sensitivity $\mathcal{O}(\log n)$ for strings of length n as was previously noted in [\[1\]](#). Let $\text{KC} : \Sigma^* \rightarrow \mathfrak{M}$ be the Kolmogorov compression where $\mathfrak{M}$ is the set of all Turing machines. For a string $w \in \Sigma^n$, suppose that $\text{KC}(w) = M \in \mathfrak{M}$.

Then for a string $w' \sim w$ that differs on the i^{th} bit, one can obtain M' which outputs w' by (1) running M to obtain w , and (2) changing the i^{th} character of w to obtain w' . Then the description of M' needs the description of M and i plus some constant. Since it takes $\log n$ bits to encode i , It follows that $\text{KC}(w') \leq |M'| \leq |M| + \mathcal{O}(\log n) = \text{KC}(w) + \mathcal{O}(\log n)$. In particular, the global sensitivity of Kolmogorov compression is upper bounded $\mathcal{O}(\log n)$. While Kolmogorov compression is not computable, when it comes to efficient compression ratios, Kolmogorov compression is at least as effective as any other compression scheme, i.e., for any compression scheme **Compress** there is a universal constant C such that $|\text{KC}(w)| \leq C + |\text{Compress}(w)|$ for all string $w \in \Sigma^*$.⁷ Thus, the goal of designing a compression algorithm with low global sensitivity does not need to be inconsistent with the goal of designing a compression algorithm that achieves good compression rates.

We note that one can construct a *computable* variant of Kolmogorov compression **CKC** that preserves low global sensitivity and is competitive with any efficiently computable compression algorithm although the compression algorithm **CKC** itself is computationally inefficient. Instead of outputting the minimum-size Turing machine, one can output the *minimum-score* Turing machine M followed by $1 \circ 0^{\log t_M}$ (where t_M is the running time of the machine M), where the *score* of a Turing machine M is defined as $\text{Score}(M) := |M| + 1 + \log t_M$.⁸ Since the length of the compression becomes the score of the minimum-score Turing machine, one can similarly argue that this computable variant of Kolmogorov compression has global sensitivity $\mathcal{O}(\log n)$. To see that the compression scheme is computable, note that any string w of length n will have compression length at most $\mathcal{O}(n + \log n)$, i.e., the machine M_w which hardcodes w and copies it to the output tape in $\mathcal{O}(n)$ has $\text{Score}(M_w) = \mathcal{O}(n + \log n)$. Thus, to find the Turing machine M minimizing $\text{Score}(M)$ we only need to enumerate Turing machines with description length at most $\mathcal{O}(n + \log n)$ and we only need to run each of these TMs for at most $2^{\mathcal{O}(n + \log n)}$ steps since machines that run longer than this cannot have minimum score. While the compression take exponential time, it is computable as the total amount of work is finite.

It is also easy to see that the compression rate for our computable variant of Kolmogorov compression is at least as good as *any* efficiently computable compression algorithm, i.e., for any compression scheme **Compress** running in time $\mathcal{O}(n^c)$ for some constant c and *any* string $w \in \Sigma^n$ we have $|\text{CKC}(w)| \leq |\text{Compress}(w)| + \log n^c + \mathcal{O}(1) = |\text{Compress}(w)| + \mathcal{O}(\log n)$. This implies that

⁷ The Turing Machine M can implement any decompression algorithm. We can hardcode $z = \text{Compress}(w)$ to obtain a new machine M_z which simulates M on the input z to recover w .

⁸ Briefly, any string w of length n will have compression length at most $\mathcal{O}(n + \log n)$, i.e., consider the machine M_w which hardcodes w and copies it to the output tape in $\mathcal{O}(n)$ steps we have $\text{Score}(M_w) = \mathcal{O}(n + \log n)$. Thus, to find the minimum score TM, we only need to enumerate Turing machines with description length at most $\mathcal{O}(n + \log n)$ and we only need to run each of these TMs for at most $2^{\mathcal{O}(n + \log n)}$ steps. This will take exponential time, but the total amount of work is still finite and hence the compression algorithm is computable.

a computable compression scheme which achieves good compression ratios does not necessarily have high global sensitivity.

Proposition 1. *Let $\text{CKC} : \Sigma^* \rightarrow \mathfrak{M} \times \{0,1\}^*$ be the computable variant of Kolmogorov compression. Then $\text{GS}_{\text{CKC}}(n) = \mathcal{O}(\log n)$.*

Proof Sketch. (See [Appendix D](#) for a more rigorous proof.) For a string $w \in \Sigma^n$, suppose that $\text{CKC}(w) = (M, 1 \circ 0^{t_M})$ where M is the minimum-score Turing Machine that outputs w running in time t_M . For $w' \sim w$ that differs on the i^{th} bit, one can obtain a Turing Machine M' that outputs w' by running M to obtain w and then changing the i^{th} character of w to obtain w' . Then the running time of M' is $t_{M'} = t_M + \mathcal{O}(n)$. Furthermore, the description of M' needs the description of M plus i plus some constant, which implies $|M'| \leq |M| + \mathcal{O}(\log n)$ since it takes $\log n$ bits to encode i .

Let $\text{CKC}(w') = (M'', 1 \circ 0^{t_{M''}})$ where M'' is the minimum-score Turing Machine that outputs w' with running time $t_{M''}$. Then by definition, we have $\text{Score}(M'') \leq \text{Score}(M')$. Hence,

$$\begin{aligned} |\text{CKC}(w')| &= |M''| + 1 + \log t_{M''} = \text{Score}(M'') \leq \text{Score}(M') = |M'| + 1 + \log t_{M'} \\ &\leq |M| + \mathcal{O}(\log n) + \log(t_M + \mathcal{O}(n)) \leq |M| + \log t_M + \mathcal{O}(\log n) \\ &= |\text{CKC}(w)| + \mathcal{O}(\log n), \end{aligned}$$

which implies $\text{GS}_{\text{CKC}}(n) = \max_{w \in \Sigma^n} \max_{w' : w \sim w'} ||\text{CKC}(w)| - |\text{CKC}(w')|| = \mathcal{O}(\log n)$. Note that $\log(t_M + \mathcal{O}(n)) \leq \max\{\log t_M, \log \mathcal{O}(n)\} + 1$ and the last inequality above holds whatever the maximum is between $\log t_M$ and $\log \mathcal{O}(n)$. $\square$

References

1. T. Akagi, M. Funakoshi, and S. Inenaga. Sensitivity of string compressors and repetitiveness measures. *Inf. Comput.*, 291(C), Mar. 2023.
2. J. Alakuijala and Z. Szabadka. Brotli Compressed Data Format. RFC 7932, July 2016.
3. H. Bannai, P. Charalampopoulos, and J. Radoszewski. Maintaining the Size of LZ77 on Semi-Dynamic Strings. In S. Inenaga and S. J. Puglisi, editors, *35th Annual Symposium on Combinatorial Pattern Matching (CPM 2024)*, volume 296 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 3:1–3:20, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
4. D. L. Black and D. McGrew. Using Authenticated Encryption Algorithms with the Encrypted Payload of the Internet Key Exchange version 2 (IKEv2) Protocol. RFC 5282, Aug. 2008.
5. J. Blocki, E. Grigorescu, and T. Mukherjee. Privately Estimating Graph Parameters in Sublinear Time. In M. Bojańczyk, E. Merelli, and D. P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, volume 229 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1–26:19, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

6. J. Blocki, E. Grigorescu, T. Mukherjee, and S. Zhou. How to Make Your Approximation Algorithm Private: A Black-Box Differentially-Private Transformation for Tunable Approximation Algorithms of Functions with Low Sensitivity. In N. Megow and A. Smith, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2023)*, volume 275 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 59:1–59:24, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
7. J. P. Degabriele. Hiding the lengths of encrypted messages via gaussian padding. In G. Vigna and E. Shi, editors, *ACM CCS 2021*, pages 1549–1565. ACM Press, Nov. 2021.
8. L. P. Deutsch. DEFLATE Compressed Data Format Specification version 1.3. RFC 1951, May 1996.
9. C. Dwork, F. McSherry, K. Nissim, and A. Smith. Calibrating noise to sensitivity in private data analysis. In S. Halevi and T. Rabin, editors, *TCC 2006*, volume 3876 of *LNCS*, pages 265–284. Springer, Berlin, Heidelberg, Mar. 2006.
10. V. A. E. Farias, F. T. Brito, C. Flynn, J. C. Machado, and D. Srivastava. Differentially private multi-objective selection: Pareto and aggregation approaches, 2025.
11. D. Fisher. CRIME Attack Uses Compression Ratio of TLS Requests as Side Channel to Hijack Secure Sessions. ThreatPost. Retrieved September 13, 2012.
12. S. Giuliani, S. Inenaga, Z. Lipták, G. Romana, M. Sciortino, and C. Urbina. Bit catastrophes for the burrows-wheeler transform. In *Developments in Language Theory: 27th International Conference, DLT 2023, Umeå, Sweden, June 12–16, 2023, Proceedings*, page 86–99, Berlin, Heidelberg, 2023. Springer-Verlag.
13. Y. Gluck, N. Harris, and A. Prado. BREACH: Reviving the CRIME Attack. <https://breachattack.com/>, 2013.
14. D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
15. G. Lagarde and S. Perifel. Lempel-ziv: a “one-bit catastrophe” but not a tragedy. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1478–1495. SIAM, 2018.
16. A. Lempel and J. Ziv. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.
17. A. Lempel and J. Ziv. Compression of individual sequences via variable-rate coding. *IEEE Transactions on Information Theory*, 24(5):530–536, 1978.
18. S. Liu, Y. Cao, T. Murakami, J. Liu, and M. Yoshikawa. CARGO: Crypto-Assisted Differentially Private Triangle Counting Without Trusted Servers . In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 1671–1684, Los Alamitos, CA, USA, May 2024. IEEE Computer Society.
19. Z. Lu, H. J. Asghar, M. A. Kaafar, D. Webb, and P. Dickinson. A differentially private framework for deep learning with convexified loss functions. *IEEE Transactions on Information Forensics and Security*, 17:2151–2165, 2022.
20. R. Palacios, A. F. Fernández-Portillo, E. F. Sánchez-Úbeda, and P. García-De-Zúñiga. Htb: a very effective method to protect web servers against breach attack to https. *IEEE Access*, 10:40381–40390, 2022.
21. B. Paulsen, C. Sung, P. A. Peterson, and C. Wang. Debreach: Mitigating compression side channels via static analysis and transformation. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 899–911. IEEE, 2019.

22. Z. Ratliff and S. Vadhan. A framework for differential privacy against timing attacks. *arXiv preprint arXiv:2409.05623*, 2024.
23. J. Rizzo and T. Duong. The CRIME attack. Presentation at Ekoparty 2012, 2012.
24. W. Sheng, J. Chen, C. Hu, B. Cai, M. Han, and J. Yu. Differentially private distance query with asymmetric noise, 2025.
25. Y. Song. Refined techniques for compression side-channel attacks. 2024. (*Master's Thesis, ETH Zürich*).
26. J. Tètek. Additive Noise Mechanisms for Making Randomized Approximation Algorithms Differentially Private. In A. Kumar and N. Ron-Zewi, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2024)*, volume 317 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 73:1–73:20, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
27. A. Ünsal and M. Önen. Chernoff information as a privacy constraint for adversarial classification. In *2024 60th Annual Allerton Conference on Communication, Control, and Computing*, pages 1–8, 2024.
28. H. Wang, D. Zhao, and H. Wang. Dynamic global sensitivity for differentially private contextual bandits. In *Proceedings of the 16th ACM Conference on Recommender Systems*, RecSys '22, page 179–187, New York, NY, USA, 2022. Association for Computing Machinery.
29. T. Welch. A technique for high-performance data compression. *Computer*, 17(6):8–19, 1984.
30. M. Wicker, P. Sosnin, I. Shilov, A. Janik, M. N. Müller, Y.-A. de Montjoye, A. Weller, and C. Tsay. Certification for differentially private prediction in gradient-based training, 2024.
31. M. Zhang, X. Liu, and L. Yin. Sensitivity estimation for differentially private query processing, 2023.

A Differentially Private Compression: Missing Proofs

Claim 8. *Let $w \sim w'$ be any strings and $\epsilon, \delta > 0$ be DP parameters. Let*

$$\text{BAD} = \begin{cases} [|\text{Compress}(w)| + 1, |\text{Compress}(w')| + 1] & \text{if } |\text{Compress}(w')| \geq |\text{Compress}(w)|; \\ \{|\text{Compress}(w)| + 1\} & \text{otherwise,} \end{cases}$$

and $\mathcal{T}(w, \epsilon, \delta) := |\text{DPCompress}(w, \epsilon, \delta)|$. Then $\Pr[\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}] \leq \delta$.

Proof. Let $w \sim w'$ be given and let $\text{BAD}' = [|\text{Compress}(w)| + 1, |\text{Compress}(w)| + \text{GS}_f + 1]$. Observe that we always have $\text{BAD} \subseteq \text{BAD}'$ since we must $|\text{Compress}(w)| + \text{GS}_f + 1 \geq \text{LCompress}(w') + 1$ and $|\text{Compress}(w)| + \text{GS}_f + 1 \geq |\text{Compress}(w)| + 1$. We first observe that $\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}'$ if and only if $Z \leq \text{GS}_f + 1 - k$. The reason is as follows:

- If $\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}'$, then $|\text{Compress}(w)| + 1 \leq |\text{Compress}(w)| + \max\{1, Z + k\} \leq |\text{Compress}(w)| + \text{GS}_f + 1$, which implies $Z + k \leq \text{GS}_f + 1$. Hence, $Z \leq \text{GS}_f + 1 - k$.
- If $Z \leq \text{GS}_f + 1 - k$, then $Z + k \leq \text{GS}_f + 1$, which implies $\max\{1, Z + k\} \leq \max\{1, \text{GS}_f + 1\} = \text{GS}_f + 1$. Hence, we observe $|\text{Compress}(w)| + 1 \leq |\text{Compress}(w)| + \max\{1, Z + k\} \leq |\text{Compress}(w)| + \text{GS}_f + 1$ and therefore $\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}'$.

Therefore,

$$\Pr[\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}] \leq \Pr[\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}'] \quad (1)$$

$$= \Pr[Z \leq \text{GS}_f + 1 - k] \quad (2)$$

$$= \Pr \left[Z \leq -\frac{\text{GS}_f}{\epsilon} \ln \left(\frac{1}{2\delta} \right) \right] \quad (3)$$

$$= \frac{1}{2} \exp \left(-\ln \left(\frac{1}{2\delta} \right) \right) \quad (4)$$

$$= \frac{1}{2} \exp(\ln(2\delta)) = \delta,$$

where Equation (1) follows because $\text{BAD} \subseteq \text{BAD}'$, Equation (2) follows by our observation that $\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}'$ if and only if $Z \leq \text{GS}_f + 1 - k$, Equation (3) follows by definition of k , and Equation (4) follows because Z is sampled from the Laplace distribution $Z \sim \text{Lap}(\frac{\text{GS}_f}{\epsilon})$. $\square$

Reminder of Theorem 2. Define $A_{\epsilon, \delta}(w) := \text{DPCompress}(w, \epsilon, \delta)$ then, for any $\epsilon, \delta > 0$, the algorithm $A_{\epsilon, \delta}$ is a (ϵ, δ) -differentially private compression scheme.

Proof. Let $w \sim w'$ be given and let (ϵ, δ) be the differential privacy parameters. Consider the set BAD as defined in Claim 8.

We first claim that $\forall t \notin \text{BAD}$, then $\Pr[\mathcal{T}(w, \epsilon, \delta) = t] \leq e^\epsilon \Pr[\mathcal{T}(w', \epsilon, \delta) = t]$. Note that if $t < |\text{Compress}(w)| + 1$ then we have $\Pr[\mathcal{T}(w, \epsilon, \delta) = t] = 0$ so the above inequality certainly holds. Otherwise, we have $t > \max\{|\text{Compress}(w)| + 1, \text{LCompress}(w') + 1\}$ since $t \notin \text{BAD}$ and we have

$$\begin{aligned} \frac{\Pr[\mathcal{T}(w, \epsilon, \delta) = t]}{\Pr[\mathcal{T}(w', \epsilon, \delta) = t]} &= \frac{\Pr[|\text{Compress}(w)| + Z_1 + k = t]}{\Pr[\text{LCompress}(w') + Z_2 + k = t]} \\ &= \frac{\frac{\epsilon}{2\text{GS}_f} \exp(\frac{-\epsilon}{\text{GS}_f} |t - |\text{Compress}(w)|| - k|)}{\frac{\epsilon}{2\text{GS}_f} \exp(\frac{-\epsilon}{\text{GS}_f} |t - \text{LCompress}(w') - k|)} \\ &= \exp \left[\frac{\epsilon}{\text{GS}_f} (|\epsilon - \text{LCompress}(w') - k| - |\epsilon - |\text{Compress}(w)|| - k|) \right] \\ &\leq \exp \left[\frac{\epsilon}{\text{GS}_f} |\text{LCompress}(w') - |\text{Compress}(w)|| \right] \\ &\leq \exp(\epsilon) = e^\epsilon, \end{aligned}$$

where Z_1 (resp. Z_2) is the random variable we sampled in $\text{DPCompress}(w, \epsilon, \delta)$ (resp. $\text{DPCompress}(w', \epsilon, \delta)$) and the last inequality follows because $||\text{Compress}(w)| - \text{LCompress}(w')| \leq \text{GS}_f$. Now let $S \subseteq \mathbb{N}$ be any subset of outcomes and note that

$$\begin{aligned} \Pr[\mathcal{T}(w, \epsilon, \delta) \in S] &= \Pr[\mathcal{T}(w, \epsilon, \delta) \in S \setminus \text{BAD}] + \Pr[\mathcal{T}(w, \epsilon, \delta) \in S \cap \text{BAD}] \\ &\leq \Pr[\mathcal{T}(w, \epsilon, \delta) \in S \setminus \text{BAD}] + \Pr[\mathcal{T}(w, \epsilon, \delta) \in \text{BAD}] \end{aligned}$$

$$\leq \Pr[\mathcal{T}(w, \epsilon, \delta) \in S \setminus \text{BAD}] + \delta \quad (5)$$

$$\begin{aligned} &= \delta + \sum_{t \in S \setminus \text{BAD}} \Pr[\mathcal{T}(w, \epsilon, \delta) = t] \\ &\leq \delta + \sum_{t \in S \setminus \text{BAD}} e^\epsilon \Pr[\mathcal{T}(w', \epsilon, \delta) = t] \quad (6) \\ &\leq \delta + e^\epsilon \Pr[\mathcal{T}(w', \epsilon, \delta) \in S \setminus \text{BAD}] \\ &\leq e^\epsilon \Pr[\mathcal{T}(w', \epsilon, \delta) \in S] + \delta, \end{aligned}$$

where Equation (5) holds by Claim 8, moreover the condition for Equation (6) holds since the previous reasoning when $\Pr[\mathcal{T}(w, \epsilon, \delta) = t] = e^\epsilon \Pr[\mathcal{T}(w', \epsilon, \delta) = t]$, $\forall t \notin \text{BAD}$. Hence, $A_{\epsilon, \delta}$ is (ϵ, δ) -differentially private. $\square$

B Upper Bound for the Global Sensitivity of the LZ77 Compression Scheme: Missing Proofs

Definition 4. Let $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression scheme and w, w' be strings of length n . Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Then we say that for $k \in [t']$, the block B'_k starts inside B_i , $i \in [t]$, if and only if $s_i \leq s'_k \leq f_i$.

We can also define a predicate $\text{StartInside} : \mathbb{Z} \times \mathbb{Z} \rightarrow \{0, 1\}$ for the blocks above, where

$$\text{StartInside}(i, k) := \begin{cases} 1 & \text{if } i \in [t] \wedge k \in [t'] \wedge s_i \leq s'_k \leq f_i, \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

That is, $\text{StartInside}(i, k) = 1$ if and only if B'_k starts inside B_i . Let $\mathcal{M}_i$ be the set of indices of blocks for compressing w' which start inside B_i defined as follows: $\mathcal{M}_i := \{k \in [t'] : \text{StartInside}(i, k) = 1\}$. We further define $\mathcal{B}_m$ to be the set of indices $i \in [t]$ (of blocks for compressing w) where the length of the set $\mathcal{M}_i$ equals m , i.e., $\mathcal{B}_m = \{i \in [t] : |\mathcal{M}_i| = m\}$.

B.1 Analyzing the Positions of Blocks

Reminder of Lemma 1. Let $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression algorithm and $w, w' \in \Sigma^n$ such that $w \sim w'$. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Then for all $i \in [t]$, either $\mathcal{M}_i = \emptyset$ or $\mathcal{M}_i = [i_1, i_2]$ for some $i_1 \leq i_2 \leq i_1 + 1$. In particular, $|\mathcal{M}_i| \leq 2, \forall i \in [t]$.

Proof. Let us consider a particular block B_i with start location s_i and finish location f_i . Let $k \in [t']$ be the smallest index such that block B'_k starts inside B_i i.e., such that $s_i \leq s'_k \leq f_i$. Note that if no such k exists then B_i contains 0 blocks and we are immediately done.

Let j be the index where w and w' differ, i.e., $w[j] \neq w'[j]$. Now we consider the following cases:

- (1) *Case 0: $f_i < j$.* In this case, we argue the following claim.

Claim 9. *If $f_i < j$, then $B_k = B'_k \forall k \leq i$.*

Proof of Claim 9. As shown in Figure 2, we observe that $w[1, f_i] = w'[1, f'_i]$ since $f_i < j$ and let j be the index of the position where $w \sim w'$, particularly, j is greater than f_i and f'_i . Hence, running the LZ77-block-based algorithm outputs the same result for both substrings, i.e., $B_k = B'_k$ for all $k \leq i$. $\square$

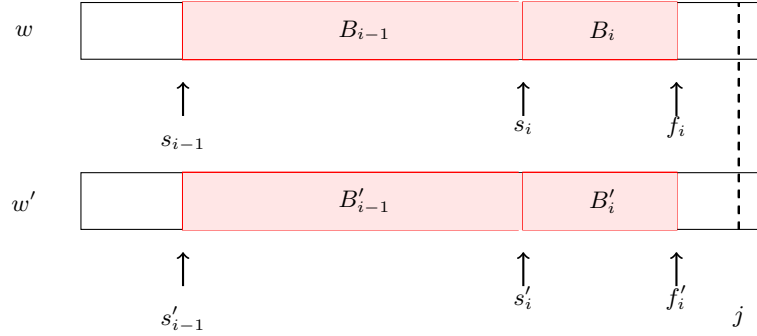


Figure 2. Compression for w and w' which $f_i < j$.

By Claim 9, we can conclude that there exists only one block B'_i that can start inside B_i .

- (2) *Case 1: $s_i \leq j \leq f_i$.* In this case, we observe the following claims:

Claim 10. *If $s_i \leq j \leq f_i$, then $s_i = s'_i$.*

Proof of Claim 10. We observe that $f_{i-1} < s_i \leq j$. Hence, by Claim 9, we have $B_k = B'_k$ for all $k \leq i-1$. This implies that $f_{i-1} = f'_{i-1}$. Hence, $s_i = f_{i-1} + 1 = f'_{i-1} + 1 = s'_i$. $\square$

Claim 11. *If $s_i \leq j \leq f_i$, then $f'_i \geq j$.*

Proof of Claim 11. Suppose for contradiction that $f'_i < j$. We first observe that $w'[s'_i, f'_i - 1]$ is the longest substring of $w'[1, s'_i - 1]$ that starts at s'_i . Next, we observe that $w[s_i, j - 1]$ is a substring of $w[1, s_i - 1]$. Note that, since j is the only index where w and w' differ, we further observe that

$$w[s_i, j - 1] = w'[s_i, j - 1] = w'[s'_i, j - 1], \quad (7)$$

where the last equality comes from Claim 10. Similarly, since $s_i \leq j$, we have

$$w[1, s_i - 1] = w'[1, s_i - 1] = w'[1, s'_i - 1]. \quad (8)$$

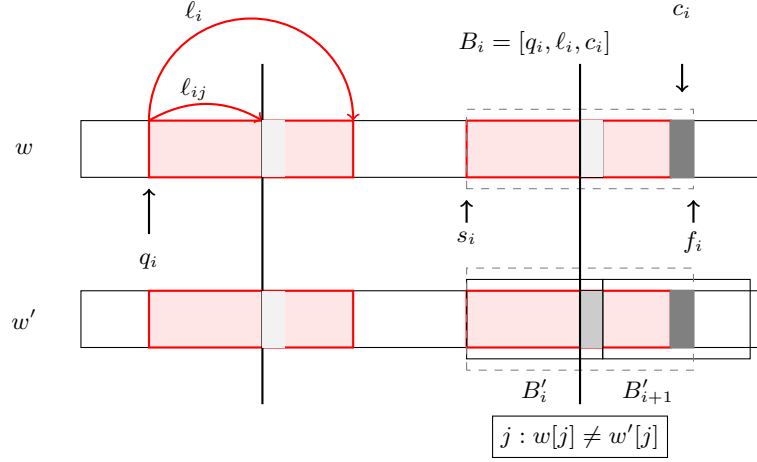


Figure 3. Compressing w and w' when the character is located at j -position such as $s_i \leq s'_i \leq f_i$ and $s_i \leq s_{k+1} \leq f_i$.

By Equation (7) and Equation (8), along with the fact that $w[s_i, j-1]$ is a substring of $w[1, s_i-1]$ that we observed before, we have that $w'[s'_i, j-1]$ is a substring of $w'[1, s'_i-1]$. Contradiction because $w'[s'_i, j-1]$ is a *longer* substring of $w'[1, s'_i-1]$ than $w'[s'_i, f'_i-1]$ starting at s'_i ! Hence, the LZ77 algorithm would have picked $w'[s'_i, j-1]$ instead of $w'[s'_i, f'_i-1]$ when constructing B'_i . This contradiction is due to the assumption that $f'_i < j$. Hence, we can conclude that $f'_i \geq j$. $\square$

Claim 12. If $s_i \leq j \leq f_i$, then $f'_{i+1} \geq f_i$.

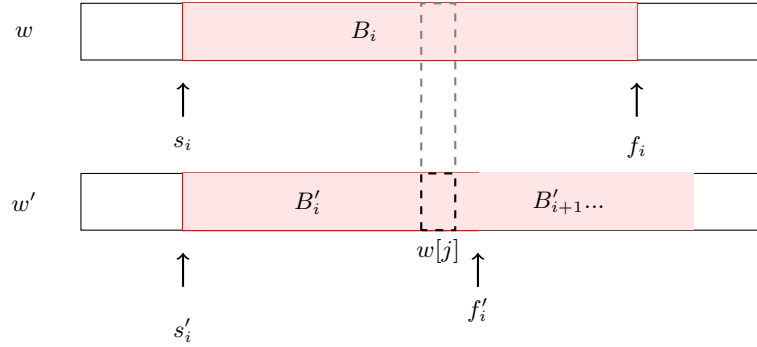


Figure 4. Compression for w and w' which $f'_{i+1} \geq j$.

Proof of Claim 12. Suppose for contradiction that $f'_{i+1} < f_i$. Observe that if $f'_{i+1} < f_i$ then we have $f'_i < s'_{i+1} < f'_{i+1} < f_i$. By definition of LZ77-Block-Based we observe that $w'[s'_{i+1}, f'_{i+1} - 1]$ was the longest possible substring of $w'[1, f'_i]$ starting at s'_{i+1} . Next, we observe that $w[j + 1, f_i - 1]$ is a substring of $w[1, f_{i-1}]$. since $f_{i-1} < j$ we further observe that:

$$w[1, f_{i-1}] = w'[1, f_{i-1}], \quad (9)$$

and

$$w[j + 1, f_i - 1] = w'[j + 1, f_i - 1]. \quad (10)$$

By Equation (9) and Equation (10), along with the fact that $w[s_i, j - 1]$ is a substring of $w[1, s_i - 1]$ that we observed before, we note that $w'[s'_{i+1}, f_i - 1]$ is a substring of $w'[1, f_{i-1}]$. However, since $f_{i-1} = f'_{i-1} < f'_i$ it follows that $w'[s'_{i+1}, f_i - 1]$ is also a substring of $w'[1, f'_i]$. This is a contradiction as $w'[s'_{i+1}, f_i - 1]$ is longer than $w'[s'_{i+1}, f'_{i+1} - 1]$ which means that LZ77-Block-Based would have selected the longer block. Thus, $f'_{i+1} \geq f_i$. $\square$

Taken together, by Claim 10, Claim 11, and Claim 12, we have $s'_i = s_i$ and $s'_{i+2} = f'_{i+1} + 1 \geq f_i + 1 > f_i$. This implies that at most two blocks B'_i and B'_{i+1} can start inside B_i . In particular, for block B'_{i+2} we have $s'_{i+2} > f'_{i+1} \geq f_i$. This completes the proof of Case 1.

- (3) *Case 2: $j < s_i$.* If $f'_k \geq f_i$ then block B'_{k+1} does not start inside block B_i as $s'_{k+1} > f'_k \geq f_i$ so that block B_i trivially contains at most 1 block. Thus, we may assume without loss of generality that $f'_k < f_i$.

Since the block B'_k starts inside B_i it is useful to define the offset $z := s'_k - s_i$. In the figure 5 can be shown:

We observe the following claims:

Claim 13. *If $j < s_i$ then $j \leq q_i + z + \ell'_k$.*

Proof of Claim 13. Suppose for contradiction that $j > q_i + z + \ell'_k$, then we note $w'[s'_k, f'_k - 1]$ is the longest possible substring of $w'[1, s'_k - 1]$ that starts at s'_k . Next we observe that $w[q_i + z, q_i + z + \ell'_k]$ is trivially a substring of $w[1, j - 1]$ since we assumed $j > q_i + z + \ell'_k$. Furthermore, we have $w[q_i + z, q_i + z + \ell'_k] = w[s_i + z, s_i + z + \ell'_k]$ since $s_i + z + \ell'_k = f'_k < f_i$. Therefore, $w[s_i + z, s_i + z + \ell'_k]$ is a substring of $w[1, j - 1]$. We further observe that

$$w[s_i + z, s_i + z + \ell'_k] = w[s'_k, f'_k] = w'[s'_k, f'_k] \quad (11)$$

and

$$w[1, j - 1] = w'[1, j - 1]. \quad (12)$$

By equation (11) and (12) we have that $w'[s'_k, f'_k]$ is a substring of $w'[1, j - 1]$, which is a substring of $w'[1, s'_k - 1]$. Contradiction because $w'[s'_k, f'_k]$ is a *longer* substring of $w'[1, s'_k - 1]$ than $w'[s'_k, f'_k - 1]$ starting at s'_k ! Hence, the LZ77 algorithm would have picked $w'[s'_k, f'_k]$ instead of $w'[s'_k, f'_k - 1]$. This contradiction is due to the assumption that $j > q_i + z + \ell'_k$. Hence, we can conclude that $j \leq q_i + z + \ell'_k$. $\square$

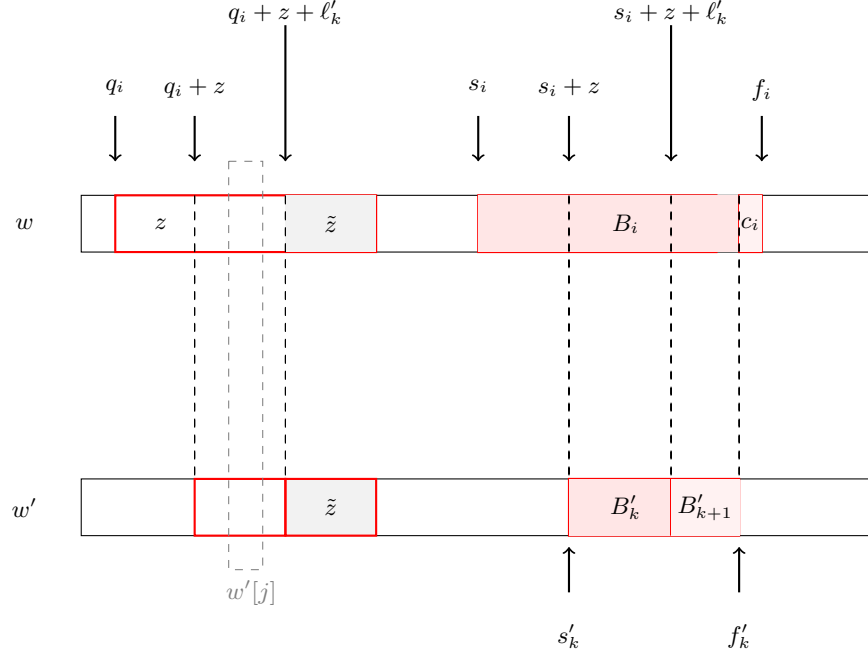


Figure 5. Compression for w and w' which $j < s_i$

Claim 14. *If $j < s_i$ then $f'_{k+1} \geq f_i$.*

Proof of Claim 14. Suppose for contradiction that $f'_{k+1} < f_i$. By the definition of the LZ77-Block-Based compression algorithm, we first observe that $w'[s'_{k+1}, f'_{k+1} - 1]$ is the longest possible substring of $w'[1, s'_{k+1} - 1]$ starting at s'_{k+1} . It is useful for our proof to define $\tilde{z} := (f_i - 1) - f'_k$. We observe that, by the definition of the LZ77-Block-Based compression algorithm,

$$w[q_i + z + \ell'_k + 1, q_i + z + \ell'_k + \tilde{z}] = w[s_i + z + \ell'_k + 1, s_i + z + \ell'_k + \tilde{z}], \quad (13)$$

since the LHS of (13) is copied to the RHS of (13) when we run the algorithm. We also observe that

$$\begin{aligned} s_i + z + \ell'_k + 1 &= s_i + (s'_k - s_i) + \ell'_k + 1 \\ &= s'_k + \ell'_k + 1 \\ &= f'_k + 1 \\ &= s'_{k+1}, \end{aligned} \quad (14)$$

and

$$\begin{aligned} s_i + z + \ell'_k + \tilde{z} &= s_i + (s'_k - s_i) + \ell'_k + (f_i - 1) - f'_k \\ &= s'_k + \ell'_k + (f_i - 1) - f'_k \end{aligned}$$

$$\begin{aligned}
&= f'_k + (f_i - 1) - f'_k \\
&= f_i - 1.
\end{aligned} \tag{15}$$

Together with Equation (14) and (15), by Equation (13) we have

$$w[q_i + z + \ell'_k + 1, q_i + z + \ell'_k + \tilde{z}] = w[s'_{k+1}, f_i - 1]. \tag{16}$$

Due to Claim 13, we have $j < q_i + z + \ell'_k + 1$. Since j was the unique index where $w[j] \neq w'[j]$, we have

$$w[q_i + z + \ell'_k + 1, q_i + z + \ell'_k + \tilde{z}] = w'[q_i + z + \ell'_k + 1, q_i + z + \ell'_k + \tilde{z}], \tag{17}$$

and

$$w[s'_{k+1}, f_i - 1] = w'[s'_{k+1}, f_i - 1]. \tag{18}$$

Applying Equations (17) and (18) to Equation (16), we obtain

$$w'[q_i + z + \ell'_k + 1, q_i + z + \ell'_k + \tilde{z}] = w'[s'_{k+1}, f_i - 1]. \tag{19}$$

Since $w'[q_i + z + \ell'_k + 1, q_i + z + \ell'_k + \tilde{z}]$ is a substring of $w'[1, s'_{k+1} - 1]$, we observe that $w'[s'_{k+1}, f_i - 1]$ is a substring of $w'[1, s'_{k+1} - 1]$ starting at s'_{k+1} . Contradiction since $f_i - 1 > f'_{k+1} - 1$, which implies the LZ77-Block-Based compression algorithm would have picked $w'[s'_{k+1}, f_i - 1]$ instead of $w'[s'_{k+1}, f'_{k+1} - 1]$ when constructing the block B'_{k+1} . This contradiction is due to the assumption $f'_{k+1} < f_i$. Hence, we can conclude that $f'_{k+1} \geq f_i$.

Taken together, by Claim 13 and Claim 14, we have $j \leq q_i + z + \ell'_k$ and $f'_{k+1} \geq f_i$. this implies that at most two blocks B'_k and B'_{k+1} can start inside B_i . In particular, for block B'_{k+2} we have $s'_{k+2} \geq f'_{k+1} \geq f_i$. This completes the proof of Case 2. $\square$

We have considered Lemma 1, where in Case 0 we showed that if $f_i < j$, then $B_k = B'_k \forall k \leq i$ what means, in fact, for Claim 9 there is only a block B'_i that start inside. In Case 1 we showed in Claim 10 that If $s_i \leq j \leq f_i$, then $s_i = s'_i$, that explains when both start position are the same, so then there is only one block that can start inside B_i . However, we showed in Claim 11 that if $s_i \leq j \leq f_i$, then $f'_i \geq j$ what means there are two blocks B'_i and B'_{i+1} that start inside B_i . Eventually, by Claim 12 if $s_i \leq j \leq f_i$, then $f'_{i+1} \geq f_i$ and this condition implies immediately since $s_i = s'_i$ that at most two blocks B'_i and B'_{i+1} can start inside B_i . In Case 2, we proved in Claim 13 that if $j < s_i$ then $j \leq q_i + z + \ell'_k$ and in Claim 14 if $j < s_i$ then $f'_{k+1} \geq f_i$. We established that taking all possible assumptions of our lemma are valid. By proving that *at most two blocks can start inside B_i* holds in all scenarios, we have proved that the lemma is true in general. Therefore, we conclude that Lemma 1 is proven. $\square$

Claim 15. Let $\text{Compress} : (\Sigma)^* \rightarrow (\Sigma')^*$ be the LZ77 compression function and w, w' be strings of length n and $w \sim w'$. Then $||\text{Compress}(w)| - |\text{Compress}(w')|| \leq t_2(2 \lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$.

Proof of Claim 15. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. We first observe that there are t blocks in $\text{Compress}(w)$ and encoding each block we need $2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil$ bits, since each block consists of two integers q_i and ℓ_i , and one character c_i with $0 \leq q_i, \ell_i \leq n-1$ and $c_i \in \Sigma$. Hence, $|\text{Compress}(w)| = t(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$, and $|\text{Compress}(w')| = t'(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$. Then, $|\text{Compress}(w)| - |\text{Compress}(w')| = |t - t'| (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$. By Claim 1 above, we observe that $|t - t'| \leq t_2$. Hence, we have $||\text{Compress}(w)| - |\text{Compress}(w')|| \leq t_2(2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil)$. $\square$

B.2 Counting Type-2 Blocks for LZ77

Reminder of Lemma 2. Let $\text{Compress} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression function and w, w' be strings of length n and $w \sim w'$. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. Then $t_2 \leq \frac{\sqrt[3]{9}}{2} n^{2/3} + \frac{\sqrt[3]{3}}{2} n^{1/3} + 1$.

Proof of Lemma 2. To prove Lemma 2, we need the following claims.

Claim 16. If $i \in \mathcal{B}_2$ then either (1) $s_i \leq j \leq f_i$ or (2) $j - \ell_i < q_i \leq j$.

Proof. We first observe that it has to be either $j \geq s_i$ or $j < s_i$.

- If $j \geq s_i$, then we want to show that $s_i \leq j \leq f_i$. Suppose for contradiction that $j > f_i$. Then we have $w[1, f_i] = w'[1, f_i]$, i.e., two substrings are identical. Hence, by definition of the LZ77 compression algorithm, the block constructions are also the same, i.e., $s_i = s'_i$ and $f_i = f'_i$, where s'_i and f'_i denote the starting and finishing index for the block B'_i for compressing w' . This implies that $\mathcal{M}_i = \{i\}$ and $i \in \mathcal{B}_1$. Contradiction since $i \in \mathcal{B}_2$. Hence, we have $s_i \leq j \leq f_i$ if $j \geq s_i$.
- If $j < s_i$, then we want to show that $j - \ell_i < q_i \leq j$, i.e., $q_i \leq j < q_i + \ell_i$. Suppose for contradiction that $j \notin [q_i, q_i + \ell_i - 1]$. Then we have

$$w[s_i, f_i - 1] = w[q_i, q_i + \ell_i - 1] = w'[q_i, q_i + \ell_i - 1] = w'[s_i, f_i - 1]. \quad (20)$$

Let k be the smallest element in $\mathcal{M}_i$, i.e., $s_i \leq s'_k \leq f_i$ but $s'_{k-1} < s_i$.

- If $s'_k = f_i$, then if there is another $k' \in \mathcal{M}_i$ with $k' \neq k$, then by choice of k , we have $k < k'$ and therefore $s'_{k'} > f'_k > s'_k = f_i$. Hence, $|\mathcal{M}_i| = 1$ and $i \in \mathcal{B}_1$. Contradiction!
- If $s_i \leq s'_k \leq f_i - 1$, then from Equation (20) and from the fact that $f_i = s_i + \ell_i$, we observe that the substring $w'[s_i, f_i - 1]$ can be obtained by shifting the substring $w'[q_i, q_i + \ell_i - 1]$ by $s_i - q_i$. Hence,

$$\begin{aligned} w'[s'_k, f_i - 1] &= w'[s'_k, s_i + \ell_i - 1] \\ &= w'[s'_k - (s_i - q_i), s_i + \ell_i - 1 - (s_i - q_i)] \\ &= w'[q_i + (s'_k - s_i), q_i + \ell_i - 1], \end{aligned}$$

which implies that $f'_k \geq f_i$ by definition of the LZ77 compression algorithm. If there is another $k' \in \mathcal{M}_i$ with $k' \neq k$, then by choice of k , we have $k < k'$ and therefore $s'_{k'} > f'_k \geq f_i$. Hence, $|\mathcal{M}_i| = 1$ and $i \in \mathcal{B}_1$. Contradiction!

Hence, we have $j \in [q_i, q_i + \ell_i - 1]$ and therefore $q_i \leq j < q_i + \ell_i$ if $j < s_i$.

Taken together, we can conclude that if $i \in \mathcal{B}_2$ then either $s_i \leq j \leq f_i$ or $q_i \leq j < q_i + \ell_i$ must hold. $\square$

Claim 17. *If $i_1, i_2 \in \mathcal{B}_2$ then $(q_{i_1}, \ell_{i_1}) \neq (q_{i_2}, \ell_{i_2})$.*

Proof. Intuitively, we prove the claim by contradiction. Suppose there exists $i_1, i_2 \in \mathcal{B}_2$ such that $(q_{i_1}, \ell_{i_1}) = (q_{i_2}, \ell_{i_2})$. Then for the first block of $\text{Compress}(w')$ that starts inside B_{i_2} , we can copy a substring until at least the end of B_{i_2} either from $w'[q_{i_2}, q_{i_2} + \ell_{i_2} - 1]$ or from B_{i_1} since at least one of them does not contain the differing index j by Claim 16. We remark that the same intuition applies for the LZ77 with self-referencing as well — see Claim 20 in Appendix B.3 for details. Below we give the formal proof of Claim 17.

Suppose for contradiction that there exist distinct $i_1, i_2 \in \mathcal{B}_2$ such that $(q_{i_1}, \ell_{i_1}) = (q_{i_2}, \ell_{i_2})$. Without loss of generality, let $i_1 < i_2$. Now, we consider the following cases:

- *Case 1:* $j < s_{i_1}$. Since $i_1 < i_2$, we observe that $j < s_{i_1} < s_{i_2}$. Let $k \in \mathcal{M}_{i_2}$ be the *smallest* element in $\mathcal{M}_{i_2}$. Since $q_{i_1} = q_{i_2}$ and $\ell_{i_1} = \ell_{i_2}$ and $j < s_{i_1}$, we have $w[q_{i_1}, q_{i_1} + \ell_{i_1} - 1] = w[s_{i_1}, f_{i_1} - 1] = w'[s_{i_1}, f_{i_1} - 1]$ and $w[q_{i_2}, q_{i_2} + \ell_{i_2} - 1] = w[s_{i_2}, f_{i_2} - 1] = w'[s_{i_2}, f_{i_2} - 1]$. Therefore, we have

$$w'[s'_k, f_{i_2} - 1] = w'[s_{i_1} + (s'_k - s_{i_2}), f_{i_1} - 1]. \quad (21)$$

We claim $f'_k \geq f_{i_2}$. Suppose for contradiction that $f'_k < f_{i_2}$. By definition of LZ77, $w'[s'_k, f'_k - 1]$ is the longest substring of $w'[1, s'_k - 1]$ starting at s'_k . However, Equation (21) implies that $w'[s'_k, f_{i_2} - 1]$ is also a substring of $w'[1, s'_k - 1]$ which is *longer than* $w'[s'_k, f'_k - 1]$, which is a contradiction. Hence, we have $f'_k \geq f_{i_2}$.

If there is another $k' \in \mathcal{M}_{i_2}$ such that $k' \neq k$ then, by choice of k , we have $k < k'$ and therefore $s'_{k'} > f'_k \geq f_{i_2}$. This contradicts the definition of $\mathcal{M}_{i_2}$ since block $B'_{k'}$ does not start inside block B_{i_2} . It follows that $|\mathcal{M}_{i_2}| \leq 1$ and therefore $i_2 \in \mathcal{B}_1$. This is a contradiction!

- *Case 2:* $j \geq s_{i_1}$. We then observe $s_{i_1} \leq j < f_{i_1}$ (if $f_{i_1} \leq j$ then only one block would have started inside B_{i_1} by observing $w[s_{i_1}, f_{i_1} - 1] = w'[s_{i_1}, f_{i_1} - 1]$, hence $i \in \mathcal{B}_1$, which is a contradiction since $i \in \mathcal{B}_2$). And since we assumed $i_1 < i_2$, we further have $s_{i_1} \leq j < f_{i_1} < s_{i_2}$. Now since $s_{i_2} > j$, by Claim 16, we have $q_{i_2} \leq j < q_{i_2} + \ell_{i_2}$. Since $q_{i_1} = q_{i_2}$ and $\ell_{i_1} = \ell_{i_2}$ by our assumption, we then have $q_{i_1} \leq j < q_{i_1} + \ell_{i_1}$. This is a contradiction since $s_{i_1} \leq j < f_{i_1}$!

Taken together, we can conclude that if $i_1, i_2 \in \mathcal{B}_2$ then $(q_{i_1}, \ell_{i_1}) \neq (q_{i_2}, \ell_{i_2})$. $\square$

Claim 18. *Let $\mathcal{B}_2^\ell := \{i \in \mathcal{B}_2 : \ell_i = \ell\}$ and suppose that $s_{i^*} \leq j \leq f_{i^*}$ for some $i^* \in [t]$. Then $|\mathcal{B}_2^\ell| \leq \ell$ for all $\ell \neq \ell_{i^*}$, and $|\mathcal{B}_2^{\ell_{i^*}}| \leq \ell_{i^*} + 1$.*

Proof. We first observe from [Claim 16](#) that if $i \in \mathcal{B}_2$, either $s_i \leq j \leq f_i$ or $j - \ell_i < q_i \leq j$ holds. Let $i^* \in [t]$ be the unique index such that $s_{i^*} \leq j \leq f_{i^*}$.

- If $\ell \neq \ell_{i^*}$, then for all $i \in \mathcal{B}_2^\ell$, we have $j - \ell_i < q_i \leq j$ since $j \notin [s_i, f_i]$ for all $i \in \mathcal{B}_2^\ell$. Suppose for contradiction that there exists some $\ell' \neq \ell_{i^*}$ such that $|\mathcal{B}_2^{\ell'}| \geq \ell' + 1$. Then we have at least $\ell' + 1$ i 's such that $j - \ell' < q_i \leq j$. Then by the pigeonhole principle, there exists some $i_1, i_2 \in \mathcal{B}_2^{\ell'}$ such that $q_{i_1} = q_{i_2}$, which implies that $(q_{i_1}, \ell_{i_1} = \ell') = (q_{i_2}, \ell_{i_2} = \ell')$. Contradiction by [Claim 17](#)! Hence, $|\mathcal{B}_2^\ell| \leq \ell$ for all $\ell \neq \ell_{i^*}$.
- If $\ell = \ell_{i^*}$, then we clearly see $i^* \in \mathcal{B}_2^{\ell_{i^*}}$. Then what is remained to show is that $|\mathcal{B}_2^{\ell_{i^*}} \setminus \{i^*\}| \leq \ell_{i^*}$. For all $i \in \mathcal{B}_2^{\ell_{i^*}} \setminus \{i^*\}$, we have $j - \ell_i < q_i \leq j$ since $s_{i^*} \leq j \leq f_{i^*}$. Now by the previous case analysis, we have $|\mathcal{B}_2^{\ell_{i^*}} \setminus \{i^*\}| \leq \ell_{i^*}$. Hence, $|\mathcal{B}_2^{\ell_{i^*}}| \leq \ell_{i^*} + 1$. $\square$

Let's get back to the proof of [Lemma 2](#). We first observe that block B_i has length $\ell_i + 1$ and the string length is n . Hence, we have $\sum_{i \in \mathcal{B}_2} (\ell_i + 1) \leq n$. Furthermore, we observe that for $i \in \mathcal{B}_2^\ell$, the length of block B_i is $\ell + 1$. Therefore,

$$\sum_{i \in \mathcal{B}_2} (\ell_i + 1) = \sum_{\ell} \sum_{i \in \mathcal{B}_2^\ell} (\ell + 1) = \sum_{\ell} |\mathcal{B}_2^\ell| (\ell + 1) \leq n.$$

Let $x_\ell := |\mathcal{B}_2^\ell|$. Observe that $t_2 \leq \sum_{\ell} x_\ell$. Thus, to bound t_2 we want to maximize $\sum_{\ell} x_\ell$ subject to the constraints that (here, $i^* \in [t]$ is the *unique* index where $s_{i^*} \leq j \leq f_{i^*}$)

- (1) $x_\ell \leq \ell$ for all $\ell \neq \ell_{i^*}$ and $x_{\ell_{i^*}} \leq \ell_{i^*} + 1$, and
- (2) $\sum_{\ell} x_\ell (\ell + 1) \leq n$.

It is clear that the maximum is obtained by setting x_ℓ the maximum possible value for all $\ell \leq z$ (set $x_\ell = \ell$ for all $\ell \leq z$ with $\ell \neq \ell_{i^*}$, and set $x_{\ell_{i^*}} = \ell_{i^*} + 1$ if $\ell_{i^*} \leq z$) and $x_\ell = 0$ for all $\ell > z$ for some threshold value z . It is followed by a simple swapping argument. Let $c_\ell = \ell$ if $\ell \neq \ell_{i^*}$ and $c_{\ell_{i^*}} = \ell_{i^*} + 1$. Suppose we have a feasible solution $(x_0, \dots, x_{n-1})$ which satisfies the constraints and we have $0 < x_{\ell_1} < c_{\ell_1}$ and $0 < x_{\ell_2} < c_{\ell_2}$ with $\ell_1 < \ell_2$. Now define a new set of solution $(x'_0, \dots, x'_{n-1})$ where $x'_{\ell_1} = x_{\ell_1} + 1$, $x'_{\ell_2} = x_{\ell_2} - 1$, and $x'_i = x_i$ for all $i \neq \ell_1, \ell_2$. Then we observe that $0 \leq x'_\ell \leq c_\ell$ for all $\ell = 0, 1, \dots, n-1$, and furthermore,

$$\begin{aligned} \sum_{\ell} x'_\ell (\ell + 1) &= x'_{\ell_1} (\ell_1 + 1) + x'_{\ell_2} (\ell_2 + 1) + \sum_{\ell \neq \ell_1, \ell_2} x'_\ell (\ell + 1) \\ &= (x_{\ell_1} + 1)(\ell_1 + 1) + (x_{\ell_2} - 1)(\ell_2 + 1) + \sum_{\ell \neq \ell_1, \ell_2} x_\ell (\ell + 1) \\ &= \sum_{\ell} x_\ell (\ell + 1) + (\ell_1 - \ell_2) < \sum_{\ell} x_\ell (\ell + 1) \leq n, \end{aligned}$$

which implies that $(x'_0, \dots, x'_{n-1})$ is also a feasible solution. We can repeat these swaps to keep the solution feasible but with a higher value in x_ℓ for a smaller

index ℓ , and finally, with having x_ℓ the maximum possible value for all $\ell \leq z$ for some threshold z . Note that x_z could be less than z , i.e., for the threshold z , we have $x_z = c$ for some $0 < c \leq z$.

To find the threshold z , we first observe that $\left\lceil \sum_{\ell=0}^{z-1} \ell(\ell+1) \right\rceil + x_z(z+1) = \sum_{\ell \geq 0} x_\ell(\ell+1) \leq n$ by condition (2). Simplifying the sum we have $\left\lceil \sum_{\ell=0}^{z-1} \ell(\ell+1) \right\rceil + x_z(z+1) = \frac{1}{3}(z-1)z(z+1) + x_z(z+1) \geq z^3/3$ where the last inequality follows because $x_z \geq 1$. Thus, we have $z^3 \leq 3n$ and $z \leq \sqrt[3]{3n}$. Setting this value of z , we have

$$t_2 \leq \sum_{\ell} x_\ell \leq \left(\sum_{\ell=0}^{\sqrt[3]{3n}} \ell \right) + 1 = \frac{\sqrt[3]{9}}{2} n^{2/3} + \frac{\sqrt[3]{3}}{2} n^{1/3} + 1,$$

where the addition of 1 in the inequality above comes from the case if $\ell_{i^*} \leq z$. $\square$

B.3 Counting Type-2 Blocks for LZ77 with Self-Referencing

For the LZ77 compression scheme with self-referencing, we can prove an analogous claim as stated in [Lemma 4](#).

Lemma 4. *Let $\text{Compress}_{\text{SR}} : \Sigma^* \rightarrow (\Sigma')^*$ be the LZ77 compression function with self-referencing and $w \sim w'$ be strings of length n . Let $(B_1, \dots, B_t) \leftarrow \text{Compress}_{\text{SR}}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}_{\text{SR}}(w')$. Then $t_2 \leq \frac{\sqrt[3]{9}}{2} n^{2/3} + \frac{\sqrt[3]{3}}{2} n^{1/3} + 1$.*

Following a similar proof strategy to [Section 4.2](#), we verify that the same claims hold for LZ77 with self-referencing as well.

Claim 19. *For LZ77 with self-referencing, if $i \in \mathcal{B}_2$ then either (1) $s_i \leq j \leq f_i$ or (2) $j - \ell_i < q_i \leq j$.*

Proof. Since the only difference between the non-overlapping LZ77 and the LZ77 with self-referencing is that we allow for $\ell_i + q_i - 1 > \text{ctc}$ and this does not affect the proof at all, we can follow the exact same proof of [Claim 16](#). $\square$

Claim 20. *For LZ77 with self-referencing, if $i_1, i_2 \in \mathcal{B}_2$ then $(q_{i_1}, \ell_{i_1}) \neq (q_{i_2}, \ell_{i_2})$.*

Proof. We can follow a similar proof strategy with [Claim 17](#), except that since we are considering LZ77 with self-referencing, in Case 1, $w'[s'_k, f'_k - 1]$ becomes the longest substring of $w'[1, f'_k - 1]$. However, [Equation \(21\)](#) implies that $w'[s'_k, f_{i_2} - 1]$ is also a substring of $w'[1, s'_k - 1]$, which is a substring of $w'[1, f'_k - 1]$. This is a contradiction since $w'[s'_k, f'_k - 1]$ does *not* become the longest substring of $w'[1, f'_k - 1]$. Hence, we have the same conclusion that $f'_k \geq f_{i_2}$, and the rest of the proof follows the same as [Claim 17](#). $\square$

Claim 21. *For LZ77 with self-referencing, let $\mathcal{B}_2^\ell := \{i \in \mathcal{B}_2 : \ell_i = \ell\}$ and suppose that $s_{i^*} \leq j \leq f_{i^*}$ for some $i^* \in [t]$. Then $|\mathcal{B}_2^\ell| \leq \ell$ for all $\ell \neq \ell_{i^*}$, and $|\mathcal{B}_2^{\ell_{i^*}}| \leq \ell_{i^*} + 1$.*

Proof. Since the proof of [Claim 18](#) relied on [Claim 16](#) and [Claim 17](#) along with the pigeonhole principle, we can follow the same proof strategy relying on [Claim 19](#) and [Claim 20](#) instead. $\square$

Proof of Lemma 4. Since the proof of [Lemma 2](#) did not exploit the restriction that the non-overlapping LZ77 has, we can follow the same proof of [Lemma 2](#). $\square$

Theorem 6. *Let $\text{Compress}_{\text{SR}} : (\Sigma)^* \rightarrow (\Sigma')^*$ be the LZ77 compression function with self-referencing, with unbounded sliding window size $W = n$. Then*

$$\text{GS}_{\text{Compress}_{\text{SR}}} \leq \left(\frac{\sqrt[3]{9}}{2} n^{2/3} + \frac{\sqrt[3]{3}}{2} n^{1/3} + 3 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) = \mathcal{O}(n^{2/3} \log n).$$

Proof. As discussed in [Section 4.1](#), we see that [Lemma 1](#) essentially holds similarly for the self-referencing case except that there might be one type-3 block (i.e., $t_3 \leq 1$), i.e., the block containing the edited position j , as stated in [\[1, p.18\]](#). Thus, we have $t = t_0 + t_1 + t_2 + t_3$ and we further observe that $t' = \sum_{m=0}^3 m \cdot t_m = t_1 + 2t_2 + 3t_3$ which is similar to [Claim 1](#). Hence, $t' - t = 2t_3 + t_2 - t_0 \leq t_2 + 2t_3 \leq t_2 + 2$. Hence, by [Lemma 4](#),

$$\begin{aligned} ||\text{Compress}(w) - \text{Compress}(w')|| &= |t - t'| (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) \\ &\leq \left(\frac{\sqrt[3]{9}}{2} n^{2/3} + \frac{\sqrt[3]{3}}{2} n^{1/3} + 3 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil). \end{aligned}$$

Since this inequality hold for arbitrary $w \sim w'$ of length n , we have

$$\begin{aligned} \text{GS}_{\text{Compress}} &= \max_{w \in \Sigma^n} \max_{w' \in \Sigma^n \text{ s.t. } w \sim w'} ||\text{Compress}(w) - \text{Compress}(w')|| \\ &\leq \left(\frac{\sqrt[3]{9}}{2} n^{2/3} + \frac{\sqrt[3]{3}}{2} n^{1/3} + 3 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) = \mathcal{O}(n^{2/3} \log n). \quad \square \end{aligned}$$

B.4 Bounded Sliding Window

Reminder of Claim 2. *Let $\text{Compress} : (\Sigma)^* \rightarrow (\Sigma')^*$ be the LZ77 compression function with sliding window size W and $w \sim w'$ be strings of length n with $w[j] \neq w'[j]$. Let $(B_1, \dots, B_t) \leftarrow \text{Compress}(w)$ and $(B'_1, \dots, B'_{t'}) \leftarrow \text{Compress}(w')$. If $B_i \in \mathcal{B}_2$ then $s_i \leq j + W$.*

Proof. Let $B_i = [q_i, \ell_i, c_i]$ such that $s_i > j + W$. Then we will show that $B_i \in \mathcal{B}_0 \cup \mathcal{B}_1$.

- If $B_i \in \mathcal{B}_0$ then we clearly have $B_i \in \mathcal{B}_0 \cup \mathcal{B}_1$.
- If $B_i \notin \mathcal{B}_0$ then let B'_k be the first block that starts inside B_i . We argue that $f'_k \geq f_i$, which implies that $B_i \in \mathcal{B}_1$. Suppose for contradiction that $f'_k < f_i$. Then we have that $w'[s'_k, f'_k - 1]$ is the longest substring of $w'[\max\{1, s'_k - W\}, s'_k - 1]$ that starts at s'_k . Since $s_i > j + W$, we have that $j < s_i - W$

and therefore $w[q_i, q_i + \ell_i - 1] = w'[q_i, q_i + \ell_i - 1]$ since we only look at the previous W characters when finding the longest substring to construct B_i . Hence,

$$\begin{aligned} w'[q_i + (s'_k - s_i), q_i + \ell_i - 1] &= w[s_i + (s'_k - s_i), f_i - 1] \\ &= w[s'_k, f_i - 1] \\ &= w'[s'_k, f_i - 1]. \end{aligned}$$

Furthermore, since $q_i \geq s_i - W$ by definition of LZ77, we observe $q_i + (s'_k - s_i) \geq s'_k - W$, meaning that $w'[s'_k, f_i - 1]$ is also a substring of $w'[\max\{1, s'_k - W\}, s'_k - 1]$ that starts at s'_k . But since $f'_k < f_i$, $w'[s'_k, f_i - 1]$ is a *longer* substring of $w'[\max\{1, s'_k - W\}, s'_k - 1]$ that starts at s'_k than $w'[s'_k, f'_k - 1]$. Contradiction! This contradiction is due to the assumption that $f'_k < f_i$. Hence, we have $f'_k \geq f_i$ and therefore $B_i \in \mathcal{B}_1$. Therefore, $B_i \in \mathcal{B}_0 \cup \mathcal{B}_1$.

Finally, if $B_i \in \mathcal{B}_2$ then $B_i \notin \mathcal{B}_0 \cup \mathcal{B}_1$, hence we have $s_i \leq j + W$. $\square$

Theorem 7. Let $\text{Compress}_{\text{SR}} : (\Sigma)^* \rightarrow (\Sigma')^*$ be the LZ77 compression function with self-referencing, with sliding window size W . Then

$$\text{GS}_{\text{Compress}_{\text{SR}}} \leq \left(\frac{\sqrt[3]{81}}{2} W^{2/3} + \frac{\sqrt[3]{9}}{2} W^{1/3} + 5 \right) (2\lceil \log n \rceil + \lceil \log |\Sigma| \rceil) = \mathcal{O}(W^{2/3} \log n).$$

C Lower Bound for the Global Sensitivity of the LZ77 Compression Scheme: Missing Proofs

Reminder of Claim 3. Let $m \in \mathbb{N}$ and $(w, w') \leftarrow \text{QuinStr}(m)$. Then $|w| = |w'| = \Theta(m^3 \log m)$. In particular, for $m \geq 4$, $\frac{2}{3}m^3 \lceil \log m \rceil < |w| = |w'| < m^3 \lceil \log m \rceil$.

Proof. We first observe that $|w| = |w'|$ since they only differ by one character (2 and 3 in S_w and $S_{w'}$, respectively). We then observe that $|S_w| = 4m \lceil \log m \rceil + 2$ since we have $4m$ encodings with length $\lceil \log m \rceil$ each along with the character 2 and 4, which has length 1 each. And similarly, for $2 \leq \ell \leq m$ and $1 \leq u \leq \ell - 1$, we observe that $|S_{\ell,u}| = 2\ell \lceil \log m \rceil + 1$. Hence,

$$\begin{aligned} |w| &= |S_w| + \sum_{\ell=2}^m \sum_{u=1}^{\ell-1} (1 + |S_{\ell,u}|) \\ &= 4m \lceil \log m \rceil + 2 + \sum_{\ell=2}^m \sum_{u=1}^{\ell-1} (2 + 2\ell \lceil \log m \rceil) \\ &= 4m \lceil \log m \rceil + 2 + \sum_{\ell=2}^m [2(\ell - 1) + 2\ell(\ell - 1) \lceil \log m \rceil] \\ &= 4m \lceil \log m \rceil + 2 + (m - 1)m + \lceil \log m \rceil \cdot \frac{2}{3} (m^3 - m) \end{aligned}$$

$$= \frac{2}{3}m^3 \lceil \log m \rceil + \frac{10}{3}m \lceil \log m \rceil + (m-1)m + 2.$$

It is clear that $|w| > \frac{2}{3}m^3 \lceil \log m \rceil$ since the term $\frac{10}{3}m \lceil \log m \rceil + (m-1)m + 2$ is always positive for $m \geq 1$. Now we consider the upper bound. For $m = 4$, we can directly show that $\frac{2}{3}m^3 \lceil \log m \rceil + \frac{10}{3}m \lceil \log m \rceil + (m-1)m + 2 = \frac{378}{3} < 128 = m^3 \lceil \log m \rceil$. For $m \geq 5$, we have $m^2 \geq 25$ and $5(m-1)m + 10 < 5(m-1)m + 5m = 5m^2 \leq m^3$, which implies

$$\begin{aligned} |w| &= \frac{2}{3}m^3 \lceil \log m \rceil + \frac{10}{3}m \lceil \log m \rceil + (m-1)m + 2 \\ &= \frac{2}{3}m^3 \lceil \log m \rceil + \frac{10}{25} \cdot \frac{25}{3}m \lceil \log m \rceil + \frac{1}{5} [5(m-1)m + 10] \\ &< \frac{2}{3}m^3 \lceil \log m \rceil + \frac{10}{25} \cdot \frac{1}{3}m^3 \lceil \log m \rceil + \frac{1}{5}m^3 \\ &< \frac{2}{3}m^3 \lceil \log m \rceil + \frac{10}{25} \cdot \frac{1}{3}m^3 \lceil \log m \rceil + \frac{1}{5}m^3 \lceil \log m \rceil = m^3 \lceil \log m \rceil. \end{aligned}$$

Taken together, we can conclude that $|w| = |w'| = \Theta(m^3 \log m)$. $\square$

Reminder of Claim 4. For any integer $m \geq 2$, the function

$$f(\ell, u) := \frac{(\ell-2)(\ell-1)}{2} + (\ell-u)$$

defined over integers ℓ and u such that $2 \leq \ell \leq m$ and $1 \leq u \leq \ell-1$ is injective, and its range is $[\frac{(m-1)m}{2}]$.

Proof. We prove this by induction on m . For the base case ($m = 2$), the domain of the function f only consists of a single element $\{(2, 1)\}$, therefore the function is clearly injective and $f(2, 1) = 1$ shows that its range is $[1] = [\frac{(m-1)m}{2}]$. Now suppose that the claim holds for some $m = m'$ and consider the case where $m = m' + 1$. By inductive hypothesis, we already know that $f(\ell, u)$ is an injective function for $2 \leq \ell \leq m'$ and $1 \leq u \leq \ell-1$ and its range is $[\frac{(m'-1)m'}{2}]$. When $\ell = m' + 1$ and $1 \leq u \leq \ell-1 = m'$, we observe that

$$f(\ell, u) = f(m' + 1, u) = \frac{(m'-1)m'}{2} + (m' + 1 - u),$$

hence the output of the function is all different for different u 's, and its range is $[\frac{(m'-1)m'}{2} + 1, \frac{(m'-1)m'}{2} + m'] = [\frac{(m'-1)m'}{2} + 1, \frac{m'(m'+1)}{2}]$. Since $[\frac{(m'-1)m'}{2}]$ and $[\frac{(m'-1)m'}{2} + 1, \frac{m'(m'+1)}{2}]$ are mutually exclusive, we can conclude that the function is still injective over integers ℓ and u such that $2 \leq \ell \leq m' + 1$ and $1 \leq u \leq \ell-1$, and its range is $[\frac{(m'-1)m'}{2}] \cup [\frac{(m'-1)m'}{2} + 1, \frac{m'(m'+1)}{2}] = [\frac{m'(m'+1)}{2}]$, which concludes the proof. $\square$

Reminder of Claim 6. $S_{\ell,u}^L \circ 4 \circ S_{\ell,u-1}^F$ does not repeat for different ℓ and u such that $3 \leq \ell \leq m$ and $2 \leq u \leq \ell-1$.

Proof. We have $S_{\ell,u}^L = E(m - u + \ell)^2$ and $S_{\ell,u-1}^F = E(m - u + 2)^2$. Since the pairs $(-u + \ell, -u + 2)$ for $3 \leq \ell \leq m$ and $2 \leq u \leq \ell - 1$ are all distinct, the claim holds. $\square$

Reminder of Claim 7. For $2 \leq \ell \leq \lfloor \frac{m}{2} \rfloor - 1$, $S_{\ell,1}^L \circ 4 \circ S_{\ell+1,\ell}$ repeats at $S_{2\ell,\ell+1}^L \circ 4 \circ S_{2\ell,\ell}^{(\ell+1)}$.

Proof. We observe that

$$\begin{aligned} S_{2\ell,\ell+1}^L \circ 4 \circ S_{2\ell,\ell}^{(\ell+1)} &= E(m - (\ell + 1) + 2\ell)^2 \circ 4 \circ E(m - \ell + 1)^2 \circ \dots \circ E(m)^2 \circ 2 \circ E(m + 1)^2 \\ &= E(m - 1 + \ell)^2 \circ 4 \circ E(m - \ell + 1)^2 \circ \dots \circ E(m)^2 \circ 2 \circ E(m + 1)^2 \\ &= S_{\ell,1}^L \circ 4 \circ S_{\ell+1,\ell}, \end{aligned}$$

which completes the proof. $\square$

D Rigorous Proof for Proposition 1

Here we give a mathematically rigorous proof of Proposition 1.

Reminder of Proposition 1. Let $\text{CKC} : \Sigma^* \rightarrow \mathfrak{M} \times \{0, 1\}^*$ be the computable variant of Kolmogorov compression. Then $\text{GS}_{\text{CKC}}(n) = \mathcal{O}(\log n)$.

Proof. For a string $w \in \Sigma^n$, suppose that $\text{CKC}(w) = (M, 1 \circ 0^{t_M})$, i.e., M is the minimum-score Turing Machine that outputs w running in time t_M . For $w' \sim w$ that differs on the i^{th} bit, one can obtain a Turing Machine M' that outputs w' as follows:

- Run M to obtain w , and
- Flip the i^{th} bit of w to obtain w' .

Then the running time of M' is $t_{M'} = t_M + \mathcal{O}(n)$, meaning that there exist a constant $C_1 > 0$ and a positive integer n_1 such that for all $n \geq n_1$ we have $t_{M'} \leq t_M + C_1 n$. Furthermore, the description of M' needs the description of M plus i plus some constant, which implies $|M'| \leq |M| + \mathcal{O}(\log n)$ since it takes $\log n$ bits to encode i . Also, this means that there exist a constant $C_2 > 0$ and a positive integer n_2 such that for all $n \geq n_2$ we have $|M'| \leq |M| + C_2 \log n$.

Let $\text{CKC}(w') = (M'', 1 \circ 0^{t_{M''}})$, i.e., M'' is the minimum-score Turing Machine that outputs w' with running time $t_{M''}$. Then by definition, we have $\text{Score}(M'') \leq \text{Score}(M')$. Hence, for all $n \geq \max\{n_1, n_2\}$ we have

$$\begin{aligned} |\text{CKC}(w')| &= |M''| + 1 + \log t_{M''} \\ &= \text{Score}(M'') \\ &\leq \text{Score}(M') \\ &= |M'| + 1 + \log t_{M'} \\ &\leq |M| + C_2 \log n + 1 + \log(t_M + C_1 n). \end{aligned}$$

Now we consider the two cases:

- (1) If $t_M \geq C_1 n$, then for all $n \geq \max\{n_1, n_2, 2\}$ we have (recall that $\log$ is base 2)

$$\begin{aligned}
|\text{CKC}(w')| &\leq |M| + C_2 \log n + 1 + \log(t_M + C_1 n) \\
&\leq |M| + C_2 \log n + 1 + \log(2t_M) \\
&= |M| + 1 + \log t_M + 1 + C_2 \log n \\
&\leq |M| + 1 + \log t_M + (C_2 + 1) \log n \\
&= |\text{CKC}(w)| + (C_2 + 1) \log n.
\end{aligned}$$

Hence, $\text{GS}_{\text{CKC}}(n) = \max_{w \in \Sigma^n} \max_{w': w \sim w'} ||\text{CKC}(w)| - |\text{CKC}(w')|| \leq (C_2 + 1) \log n$ for all $n \geq \max\{n_1, n_2, 2\}$, which implies $\text{GS}_{\text{CKC}}(n) = \mathcal{O}(\log n)$.

- (2) If $t_M < C_1 n$, then for all $n \geq \max\{n_1, n_2, 2C_1\}$ we have

$$\begin{aligned}
|\text{CKC}(w')| &\leq |M| + C_2 \log n + 1 + \log(t_M + C_1 n) \\
&\leq |M| + C_2 \log n + 1 + \log(2C_1 n) \\
&= |M| + 1 + (C_2 + 2) \log n \\
&\leq |M| + 1 + \log t_M + (C_2 + 2) \log n \\
&= |\text{CKC}(w)| + (C_2 + 2) \log n.
\end{aligned}$$

Hence, $\text{GS}_{\text{CKC}}(n) = \max_{w \in \Sigma^n} \max_{w': w \sim w'} ||\text{CKC}(w)| - |\text{CKC}(w')|| \leq (C_2 + 2) \log n$ for all $n \geq \max\{n_1, n_2, 2C_1\}$, which implies $\text{GS}_{\text{CKC}}(n) = \mathcal{O}(\log n)$. $\square$